TraceClaw Supplementary Appendix: Controlled Cohort Evidence

Anonymous Author(s)

October 2026

1 Gateway Source-Level Trace

This appendix keeps the detailed source-code and runtime evidence for the Gateway G0–G18 path. The main text gives the argument and the compact tables; this file keeps the longer audit trail so that each claim can be checked against source and observed values.

Appendix Reading Guide

The appendix follows one rule: source detail belongs here, not in the main paper. Each stage entry records its purpose, source path, R1 value, evidence status, and branch status; evidence status says how the claim is known, while branch status says what happened to the cited source branch.

- Observed: the saved runtime trace or native runtime event directly records the stage outcome or value.
- SourceDerived: pinned source code plus observed upstream values determine the branch outcome, but the saved trace does not emit a separate event for that internal decision.
- NotObserved: the item is relevant to the audit but absent from the saved trace.

Branch and execution status is recorded separately:

- Taken: R1 follows the cited source branch.
- NotReached: the branch exists in source but R1 does not reach it.
- NotApplicable: the branch does not apply to R1's observed inputs.

NotReached and NotApplicable are not evidence categories. They normally have evidence status SourceDerived when the branch outcome follows from pinned source plus observed upstream values. "Observability gap" remains descriptive prose for a missing rationale, predicate, field, or return payload; it is not a fourth evidence status.

All source anchors used for G0–G18 and the post-G18 source mapping were mechanically checked against the unmodified upstream OpenClaw source at commit `0790d9f` using `scripts/check_anchors.py`. This check validates source-anchor fidelity only. It does not independently validate runtime evidence or the semantic correctness of the experimental claims.

For R1, Gateway coverage is 19/19 and stage-outcome observation is 19/19. Every G0–G18 stage has a directly recorded runtime outcome and at least one concrete case value. This does not imply a complete record of every internal predicate, payload field, or return object; rationale

visibility remains partial. The table below summarizes the field-level gaps that appear later in the per-stage ledger.

Stage	Observed outcome	Rationale-level gap
G0	shared-auth state available	raw credential fields are not recorded; the effective device-token candidate and source are source-derived as undefined
G1	shared credential allows the connection	exact credential comparison details and rate-limit internals are not emitted
G2	final auth passes without device-token fallback	device-signature subpath is source-derived because `hasDeviceIdentity=False`
G3	`chat.send` allowed by `operator.write`	`auth_reason` is not emitted; admin shortcut exclusion is source-derived from scopes
G4	RPC envelope validates	full raw WebSocket frame and all pre-validation fields are not archived
G5	message/session fields normalize	intermediate sanitizer object is not emitted in full
G6	requested Agent resolves to `main`	absence of every alternate override source is partly source-derived
G7	Session key resolves	persistent store path, loaded entry object, `entry.sessionId`, and timing details are not emitted
G8	Agent/Session validation passes	full validation object and backing Session identity are not emitted
G9	effective Agent is `main`	fallback candidate chain is not emitted in full
G10	send policy allows the turn	exact allow sub-branch and rule rationale are not emitted
G11	duplicate-run check creates a new dispatch	internal lock/cache state is not emitted
G12	work is admitted	admission internals, latest Session details, and concrete admitted Session identity are not emitted
G13	message context is built	full constructed context object is not emitted
G14	auto-reply dispatch is entered	complete dispatch return payload is not emitted
G15	runtime context is finalized	full finalized context object is not emitted
G16	reply orchestration entered	complete reply-dispatch return object is not emitted
G17	Agent re-check passes	fallback attempt chain is not emitted in full
G18	default resolver is selected	full resolver payload and later Gateway post-return state are not emitted

Final `rationale_visibility` count for G0–G18: FULL = 0/19, PARTIAL = 19/19, and SOURCE-DERIVED = 0/19. The count is stored in artifacts/controlled/rationale-visibility-summary.csv.

Exact page placement may change after recompilation. Therefore, the main paper refers to appendix labels and stage names rather than fixed PDF page numbers. This keeps the paper stable when figures, tables, or references are edited.

Concrete R1 Runtime Values and Exact Sources

This table records the exact source locations for the concrete values used in the main text.

Variable	Concrete R1 Value	Obtained / Observed At	Exact Source
SESSION_KEY	agent:main:dashboard:trace-82fbff4483c4	**G5**: extracted as rawSessionKey; observed again in G6/G7	chat.ts:3800 G6/G7 runtime trace
SESSION_ID	NOTRECORDED	**G7/G8**: Gateway Session ID is not emitted; AR-layer Session ID is separate	session-utils.ts:1058, 1064 possible source read at chat.ts:3848--3849
MESSAGE	"How to make a cake?"	Input exists before G3; **G4**: validated; **G5**: sanitized into inboundMessage	chat.ts:3714--3739 chat.ts:3770,3784
RUN_ID	ANON-R1-RUN-ID	Generated before G3 by Control UI; **G4**: observed in runtime; **G5**: extracted as clientRunId	chat-send.ts:266--276 G4 runtime trace chat.ts:3802

G0: Connection Authentication State Resolution

Purpose: This step converts the authentication material supplied during the WebSocket connection handshake into a structured ConnectAuthState. The Gateway receives the resolved authentication configuration, handshake authentication fields, device identity status, request metadata, proxy configuration, rate-limiting state, and client IP, and passes them to resolveConnectAuthState(...). The function resolves the available shared credential, bootstrap-token candidate, and device-token candidate, invokes the initial shared-credential authorization path, performs a shared-auth probe, and returns the resulting authentication state. G0 does not necessarily make the final connection-authentication decision. For the connection immediately preceding R1, the shared-auth path is authorized and no explicit device-token candidate is used. The returned state is therefore passed to G2, where the shared-token result is accepted and the device-token fallback is not reached.

Three objects are kept separate. The raw handshake field connectAuth.deviceToken is NotObserved (NOT RECORDED). The effective deviceTokenCandidate and deviceTokenCandidateSource are undefined (None in the pseudocode), with evidence status SourceDerived: R1 records hasDeviceIdentity=false, and auth-context.ts:141--142 therefore selects the empty-object branch. Candidate resolution is NotReached. The separate emitted field hasDeviceTokenCandidate is absent and hence NotObserved; its absence does not make the effective program state unknowable.

Python-style pseudocode	Exact source / evidence
Input to the G0 main function	**Source**
`connect_auth_state = await resolveConnectAuthState(`	src/gateway/server/ws-connection/message-handler.ts:941--957
`resolvedAuth=resolved_auth,`	message-handler.ts:941--957
`connectAuth=connect_auth,`	message-handler.ts:941--957

```
   hasDeviceIdentity=has_device_identity,                    message-handler.ts:941--957
   req=req,                                                  message-handler.ts:941--957
   trustedProxies=trusted_proxies,                           message-handler.ts:941--957
   allowRealIpFallback=allow_real_ip_fallback,               message-handler.ts:941--957
   rateLimiter=rate_limiter,                                 message-handler.ts:941--957
   clientIp=client_ip                                        message-handler.ts:941--957
)                                                            message-handler.ts:941--957
```

Main G0 function — Source

```
async def resolveConnectAuthState(                           src/gateway/server/ws-connection/auth-context.ts:126--186
   resolvedAuth,                                             auth-context.ts:126--135
   connectAuth,                                              auth-context.ts:126--135
   hasDeviceIdentity,                                        auth-context.ts:126--135
   req,                                                      auth-context.ts:126--135
   trustedProxies,                                           auth-context.ts:126--135
   allowRealIpFallback,                                      auth-context.ts:126--135
   rateLimiter,                                              auth-context.ts:126--135
   clientIp                                                  auth-context.ts:126--135
):                                                           auth-context.ts:126--135
```

Step 1: Resolve shared-credential candidate

```
shared_connect_auth =                                        src/gateway/server/ws-connection/auth-context.ts:136
   resolveSharedConnectAuth(connectAuth)                     auth-context.ts:136
shared_auth_provided = bool(shared_connect_auth)             auth-context.ts:137
# helper normalizes token/password authentication            auth-context.ts:100--109
↪ material
R1: shared_auth_provided = True                              Observed:G0resolvedevent
```

Step 2: Resolve bootstrap-token candidate

```
bootstrap_token_candidate =                                  src/gateway/server/ws-connection/auth-context.ts:138--140
   normalizeOptionalString(connectAuth.bootstrapToken)       auth-context.ts:138--140
   if hasDeviceIdentity else None                            auth-context.ts:138--140
R1: hasDeviceIdentity = False                                Observed:G0startedevent
```

```
R1: bootstrap_token_candidate = None (undefined)
```
SourceDerived:hasDeviceIdentity=False;auth-context.ts:138--140

Step 3: Resolve device-token candidate

```
if hasDeviceIdentity:
```
src/gateway/server/ws-connection/auth-context.ts:141--142

```
   device_token_candidate, device_token_source =
```
auth-context.ts:141--142

```
      resolveDeviceTokenCandidate(connectAuth)
```
auth-context.ts:141--142

```
# first attempt: explicit device token
```
src/gateway/server/ws-connection/auth-context.ts:111--124

```
explicit_device_token =
```
auth-context.ts:111--124

```
   normalizeOptionalString(connectAuth.deviceToken)
```
auth-context.ts:111--124

```
if explicit_device_token:
```
auth-context.ts:111--124

```
   return explicit_device_token, "explicit-device-token"
```
auth-context.ts:111--124

```
# otherwise the helper may fall back to connectAuth.token
```
auth-context.ts:111--124

```
R1: device_token_candidate = None (undefined)
```
SourceDerived:hasDeviceIdentity=False;auth-context.ts:141--142

```
R1: explicit device-token source = NOT REACHED
```
SourceDerived:explicit-device-tokenbranchnottaken

```
R1: shared-token fallback inside this helper is not
↪ reached
```
SourceDerived:sourceplusobservedruntimevalues

Step 4: Invoke initial shared-credential authorization

```
auth_result = await authorizeWsControlUiGatewayConnect(
```
src/gateway/server/ws-connection/auth-context.ts:144--153

```
   auth=resolvedAuth,
```
auth-context.ts:145

```
   connectAuth=shared_connect_auth,
```
auth-context.ts:146

```
   req=req,
```
auth-context.ts:147

```
   trustedProxies=trustedProxies,
```
auth-context.ts:148

```
   allowRealIpFallback=allowRealIpFallback,
```
auth-context.ts:149

```
   rateLimiter=rateLimiter if shared_auth_provided else
↪ None,
```
auth-context.ts:150

```
   clientIp=clientIp
```
auth-context.ts:151

```
)
```
auth-context.ts:144--153

```
# This authorization call is represented separately as G1
↪ .
```
G1boundary

```
R1: auth_result.ok = True
```
Observed:G1result="allow"

Step 5: Probe shared-auth validity

```
shared_auth_result =
```
src/gateway/server/ws-connection/auth-context.ts:155--166
```
   await authorizeHttpGatewayConnect(
```
auth-context.ts:155--166
```
      auth=resolvedAuth with allowTailscale=False,
```
auth-context.ts:155--166
```
      connectAuth=shared_connect_auth,
```
auth-context.ts:155--166
```
      req=req,
```
auth-context.ts:155--166
```
      trustedProxies=trustedProxies,
```
auth-context.ts:155--166
```
      allowRealIpFallback=allowRealIpFallback
```
auth-context.ts:155--166
```
   )
```
auth-context.ts:155--166
```
   if shared_connect_auth else None
```
auth-context.ts:155--166
```
R1: shared credential exists, so this probe is reached
```
SourceDerived: sourceplusobservedruntimevalues

Step 6: Compute shared-auth status

```
shared_auth_ok =
```
src/gateway/server/ws-connection/auth-context.ts:170--173
```
   (shared_auth_result.ok
```
auth-context.ts:170--173
```
      and shared_auth_result.method in ["token","password
↪ "])
```
auth-context.ts:170--173
```
   or
```
auth-context.ts:170--173
```
   (auth_result.ok
```
auth-context.ts:170--173
```
      and auth_result.method == "trusted-proxy")
```
auth-context.ts:170--173
```
R1: shared_auth_ok = True
```
Observed:G0resolvedevent

Step 7: Return ConnectAuthState — Native G0 output

```
return {
```
src/gateway/server/ws-connection/auth-context.ts:175--185
```
   "authResult": auth_result,
```
auth-context.ts:176
```
   "authOk": auth_result.ok,
```
auth-context.ts:177
```
   "authMethod":
```
auth-context.ts:178--179
```
      auth_result.method
```
auth-context.ts:178--179
```
      or ("password" if resolvedAuth.mode == "password"
↪ else "token"),
```
auth-context.ts:178--179
```
   "sharedAuthOk": shared_auth_ok,
```
auth-context.ts:180
```
   "sharedAuthProvided": shared_auth_provided,
```
auth-context.ts:181

`"bootstrapTokenCandidate": bootstrap_token_candidate,`	auth-context.ts:182
`"deviceTokenCandidate": device_token_candidate,`	auth-context.ts:183
`"deviceTokenCandidateSource": device_token_source`	auth-context.ts:184
`}`	auth-context.ts:175--185

R1 concrete G0 output	Observed / derived state
`auth_result.ok = True`	Observed:G0authOk=true;G1allow
`auth_ok = True`	Observed:G0authOk=true
`auth_method = "token"`	Observed:G0resolvedevent
`shared_auth_provided = True`	Observed:G0resolvedevent
`shared_auth_ok = True`	Observed:G0resolvedevent
`bootstrap_token_candidate = None (undefined)`	SourceDerived:hasDeviceIdentity=False;auth-context.ts:138--140
`device_token_candidate = None (undefined)`	SourceDerived:hasDeviceIdentity=False;auth-context.ts:141--142
`explicit device-token source = NOT REACHED`	SourceDerived: explicit-device-tokenbranchnottaken

TraceClaw observation	Runtime evidence
`stage = "G0"`	Observed
`event = "connection_auth_state_resolved"`	Observed
`authMode = "token"`	Observed
`authMethod = "token"`	Observed
`result = "authorized"`	Observed
`hasDeviceIdentity = False`	Observed
`sharedAuthProvided = True`	Observed
`sharedAuthOk = True`	Observed
`hasBootstrapTokenCandidate = NOT RECORDED`	NotObserved:absentfromR1eventfields
`hasDeviceTokenCandidate = NOT RECORDED`	NotObserved: absentfromR1runtimeeventfields
`explicit-device-token source = NOT REACHED`	SourceDerived: explicit-device-tokenbranchnottaken

Handoff to G2	Evidence
# G0 does not terminate the connection here.	SourceDerived:sourcecontrolflow
# Effective candidate is undefined; raw device token is ↪ not recorded.	auth-context.ts:175--185

ConnectAuthState and supplied to the final-auth stage.

G0->G2boundary

R1: G2 authMethod = "token"

Observed:nextG2event

R1: G2 result = "pass"

Observed:nextG2event



G1: Initial Shared-Credential Authorization

Purpose: G1 evaluates the shared authentication credential prepared inside G0 against the Gateway's configured connection-authentication policy. The G0 authentication-state resolver invokes `authorizeWsControlUiGatewayConnect(...)` with the resolved Gateway authentication configuration, normalized shared credential, WebSocket request metadata, proxy settings, rate-limiting state, and client IP. The wrapper marks the authentication surface as `"ws-control-ui"` and delegates the decision to `authorizeGatewayConnect(...)`, which eventually enters `authorizeGatewayConnectCore(...)`. For token-mode authentication, the core path can perform rate-limit checks and then validate the supplied shared token against the configured Gateway token. G1 returns a `GatewayAuthResult`; it does not validate the device-token candidate and does not establish the final role or scopes. For the connection immediately preceding R1, TraceClaw records `authMode = "token"` and `result = "allow"`. Because R1's shared token is accepted, no denial reason is expected in the successful G1 event and no device-token fallback is needed. G2 carries the shared-token authorization forward and finalizes role and scope.

Python-style pseudocode	Exact source / evidence
Input to the G1 main function	**Source**
`auth_result = await authorizeWsControlUiGatewayConnect(`	`src/gateway/server/ws-connection/auth-context.ts:144--153`
`  auth=resolvedAuth,`	`auth-context.ts:145`
`  connectAuth=sharedConnectAuth,`	`auth-context.ts:146`
`  req=req,`	`auth-context.ts:147`
`  trustedProxies=trustedProxies,`	`auth-context.ts:148`
`  allowRealIpFallback=allowRealIpFallback,`	`auth-context.ts:149`
`  rateLimiter=rateLimiter if sharedAuthProvided else None,`	`auth-context.ts:150`
`  clientIp=clientIp,`	`auth-context.ts:151`
`  rateLimitScope=AUTH_RATE_LIMIT_SCOPE_SHARED_SECRET`	`auth-context.ts:152`
`)`	`auth-context.ts:144--153`
`R1: resolvedAuth.mode = "token"`	`Observed:G1authMode="token"`
`R1: sharedConnectAuth exists`	`Observed:G0sharedAuthProvided=True`
Main G1 function	**Source**
`async def authorizeWsControlUiGatewayConnect(params):`	`src/gateway/auth.ts:597--604`
`  return await authorizeGatewayConnect(`	`src/gateway/auth.ts:600--603`
`    **params,`	`src/gateway/auth.ts:600--603`
`    authSurface="ws-control-ui"`	`src/gateway/auth.ts:600--603`
`  )`	`src/gateway/auth.ts:600--603`
Step 1: Enter generic Gateway connection authorization	

```
async def authorizeGatewayConnect(params):
                                                        src/gateway/auth.ts:456--479
   auth = params.auth
                                                        src/gateway/auth.ts:459
   request_context = resolveGatewayAuthRequestContext(
↪ params)                                               src/gateway/auth.ts:460--461
   auth_surface = request_context.authSurface
                                                        src/gateway/auth.ts:460--461
   limiter = request_context.limiter
                                                        src/gateway/auth.ts:460--461
   ip = request_context.ip
                                                        src/gateway/auth.ts:460--461
   rate_limit_scope = request_context.rateLimitScope
                                                        src/gateway/auth.ts:460--461
   local_direct = request_context.localDirect
                                                        src/gateway/auth.ts:460--461
```

Step 2: Resolve authorization request context

```
def resolveGatewayAuthRequestContext(params):
                                                        src/gateway/auth.ts:109--126
   auth_surface = params.authSurface or "http"
                                                        src/gateway/auth.ts:113
   ip = params.clientIp
                                                        src/gateway/auth.ts:114--117
      or resolveRequestClientIp(...)
                                                        src/gateway/auth.ts:114--117
      or req.socket.remoteAddress
                                                        src/gateway/auth.ts:114--117
   return {
                                                        src/gateway/auth.ts:119--125
      "authSurface": auth_surface,
                                                        src/gateway/auth.ts:120
      "limiter": params.rateLimiter,
                                                        src/gateway/auth.ts:121
      "ip": ip,
                                                        src/gateway/auth.ts:122
      "rateLimitScope": effective_scope,
                                                        src/gateway/auth.ts:123
      "localDirect": isLocalDirectRequest(...)
                                                        src/gateway/auth.ts:124
   }
                                                        src/gateway/auth.ts:119--125
G1 wrapper: auth_surface = "ws-control-ui"
                                                        src/gateway/auth.ts:600--603
```

Step 3: Handle optional serialized rate-limit path

```
if limiter
                                                        src/gateway/auth.ts:465--475
   and shouldAllowTailscaleHeaderAuth(auth_surface)
                                                        src/gateway/auth.ts:465--475
   and auth.allowTailscale
                                                        src/gateway/auth.ts:465--475
   and not local_direct:
                                                        src/gateway/auth.ts:465--475
      return await withSerializedRateLimitAttempt(
                                                        src/gateway/auth.ts:471--475
         run=lambda: authorizeGatewayConnectCore(params)
                                                        src/gateway/auth.ts:471--475
      )
                                                        src/gateway/auth.ts:471--475
else:
                                                        src/gateway/auth.ts:478
```

```
   return await authorizeGatewayConnectCore(params)
```
src/gateway/auth.ts:478

```
# Either route ultimately enters
↪ authorizeGatewayConnectCore.
```
SourceDerived:sourcecontrolflow

```
R1: exact outer serialization branch is not recorded
```
NotObserved:rationalefieldnotemitted

Step 4: Enter core connection-auth logic

```
async def authorizeGatewayConnectCore(params):
```
src/gateway/auth.ts:481--584

```
   auth, connectAuth, req, trustedProxies = params
```
src/gateway/auth.ts:484

```
   request_context = resolveGatewayAuthRequestContext(
↪ params)
```
src/gateway/auth.ts:486--487

```
   allow_tailscale =
```
src/gateway/auth.ts:488

```
      shouldAllowTailscaleHeaderAuth(auth_surface)
```
src/gateway/auth.ts:488

Step 5: Skip authentication modes that do not apply

```
if auth.mode == "trusted-proxy":
```
src/gateway/auth.ts:490--531

```
   evaluate trusted-proxy authentication
```
src/gateway/auth.ts:490--531

```
R1: auth.mode = "token" -> branch not taken
```
SourceDerived:
sourceplusobservedruntimevalues

```
if auth.mode == "none":
```
src/gateway/auth.ts:533--535

```
   return {"ok": True, "method": "none"}
```
src/gateway/auth.ts:533--535

```
R1: branch not taken
```
SourceDerived:
sourceplusobservedruntimevalues

Step 6: Check shared-secret rate limit

```
rate_limit_result = rejectIfRateLimited(
```
src/gateway/auth.ts:537--540

```
   limiter=limiter, ip=ip, rateLimitScope=
↪ rate_limit_scope
)
```
src/gateway/auth.ts:537

src/gateway/auth.ts:537

```
if rate_limit_result:
```
src/gateway/auth.ts:538--540

```
   return rate_limit_result
```
src/gateway/auth.ts:538--540

```
# Exact R1 rate-limit state is not recorded by the G1
↪ trace.
```
NotObserved:rationalefieldnotemitted

Step 7: Evaluate optional Tailscale-header path

```
if allow_tailscale
```
src/gateway/auth.ts:542--560

```
   and auth.allowTailscale
```
src/gateway/auth.ts:542--560

```
   and not local_direct
```
src/gateway/auth.ts:542--560

```
   and not hasExplicitSharedSecretAuth(connectAuth):
```
src/gateway/auth.ts:542--560

```
      evaluate verified Tailscale user
```
src/gateway/auth.ts:548--559

```
R1: sharedConnectAuth exists                                    Observed:G0sharedAuthProvided=True

R1: Tailscale success path is therefore not used                SourceDerived:
                                                                sourceplusobservedruntimevalues
```

Step 8: Evaluate token-mode shared credential

```
if auth.mode == "token":                                        src/gateway/auth.ts:562--570
   return authorizeTokenAuth(                                   src/gateway/auth.ts:563--569
      authToken=auth.token,                                     src/gateway/auth.ts:564
      connectToken=connectAuth.token,                           src/gateway/auth.ts:565
      limiter=limiter,                                          src/gateway/auth.ts:566
      ip=ip,                                                    src/gateway/auth.ts:567
      rateLimitScope=rate_limit_scope                           src/gateway/auth.ts:568
   )                                                            src/gateway/auth.ts:563--569

R1: auth.mode = "token"                                         Observed:G1authMode="token"
```

Step 9: Validate the supplied token

```
def authorizeTokenAuth(params):                                 src/gateway/auth.ts:389--411
   if not authToken:                                            src/gateway/auth.ts:396--398
      return {"ok": False, "reason": "
↪ token_missing_config"}                                        src/gateway/auth.ts:396--398
   if not connectToken:                                         src/gateway/auth.ts:399--404
      return {"ok": False, "reason": "token_missing"}           src/gateway/auth.ts:399--404
   if not safeEqualSecret(connectToken, authToken):             src/gateway/auth.ts:405--408
      limiter.recordFailure(...)                                src/gateway/auth.ts:405--407
      return {"ok": False, "reason": "token_mismatch"}          src/gateway/auth.ts:405--408
   limiter.reset(...)                                           src/gateway/auth.ts:409
   return {"ok": True, "method": "token"}                       src/gateway/auth.ts:410

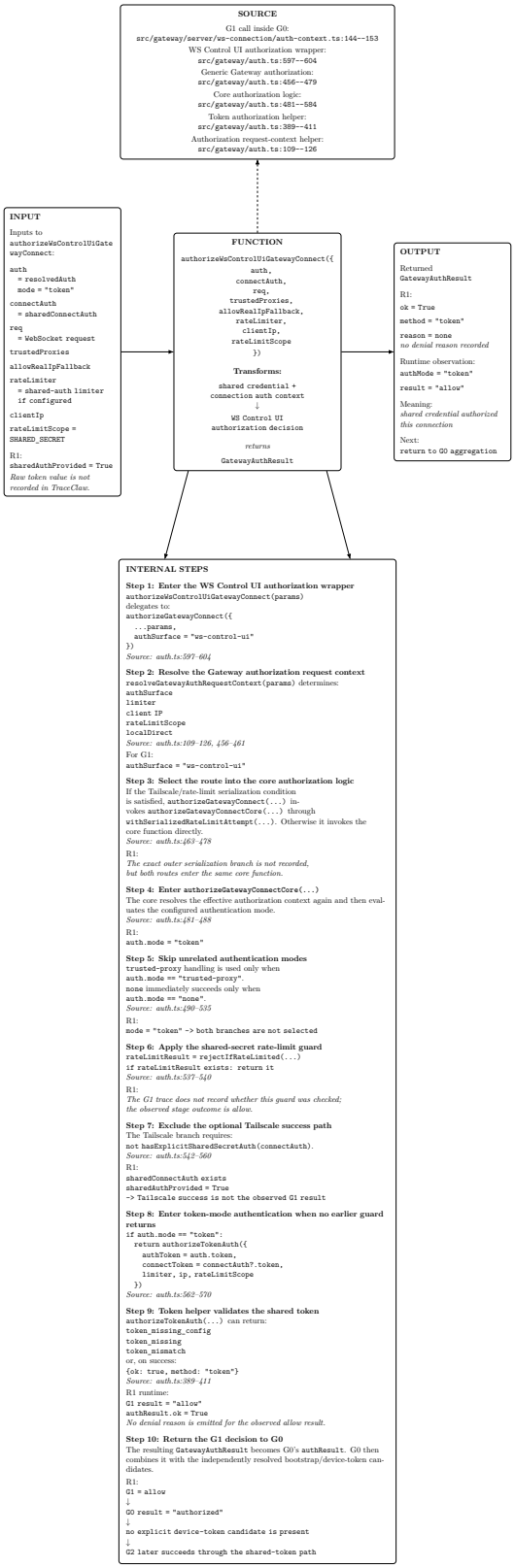
R1 runtime: G1 result = "allow"                                 Observed

# No denial reason is emitted because R1 is allowed.           Observed:auth_result.method="token";resu
                                                                lt="allow"
```

G1 output	Native / runtime result
auth_result: GatewayAuthResult	src/gateway/server/ws-connection/auth-context.ts:144
R1: auth_result.ok = True	SourceDerived:G1result="allow"
R1: auth_result.method = "token"	Observed:successfulG1tokenresult

`R1: auth_result.reason = NOT APPLICABLE`	SourceDerived:NotApplicable;R1isallowed; nodenialreasonexists
`TraceClaw stage = "G1"`	Observed
`TraceClaw event = "connection_authorization_evaluated"`	Observed
`TraceClaw authMode = "token"`	Observed
`TraceClaw result = "allow"`	Observed

Handoff	Meaning
# G1 result is returned to resolveConnectAuthState().	G1->G0aggregation
# G0 does not preserve a separate device-token candidate for R1.	src/gateway/server/ws-connection/auth-context.ts:175--185
`R1: G1 allow -> G0 authorized -> G2 shared-token path`	Observed:savedruntimeeventsequence



G2: Final Connection Authentication and Identity Finalization

Purpose: After G0 has assembled the preliminary connection-authentication state and G1 has evaluated the shared credential, G2 performs the final connection-authentication decision. The Gateway first validates the presence and cryptographic consistency of the supplied device identity. It then passes the G0 `ConnectAuthState`, device identity, requested role and scopes, rate-limiting state, and token-verification callbacks to `resolveConnectAuthDecision(...)`. This function can preserve an already successful authentication result or attempt an alternative

bootstrap-token or device-token path. After authentication succeeds, the surrounding WebSocket connection handler revalidates Gateway-auth generation, checks device pairing and approved role/scope access, constructs the authenticated Gateway client, and sends the `hello-ok` response. For the WebSocket connection carrying R1, the shared-auth path from G1 succeeds and the explicit shared-token authorization is carried into G2. The resulting connection uses `authMethod = "token"`, role `"operator"`, and scope `operator.write`. TraceClaw records `result = "pass"`.

Python-style pseudocode	Exact source / evidence
Pre-G2 device-identity gate	**Source**
`device_decision = evaluateMissingDeviceIdentity(`	src/gateway/server/ws-connection/message-handler.ts:1038--1136
`  hasDeviceIdentity=bool(device),`	message-handler.ts:1038--1136
`  role=role,`	message-handler.ts:1038--1136
`  isControlUi=is_control_ui,`	message-handler.ts:1038--1136
`  sharedAuthOk=shared_auth_ok,`	message-handler.ts:1038--1136
`  authOk=auth_ok`	message-handler.ts:1038--1136
`)`	message-handler.ts:1038--1136
`R1: hasDeviceIdentity = False`	Observed:G0/G2runtime
`R1: missing-device rejection is not taken`	SourceDerived:continuedexecution
Step 1: Validate device identity and signature material	
`derived_id = deriveDeviceIdFromPublicKey(device.publicKey ↪ )`	src/gateway/server/ws-connection/message-handler.ts:1137--1213
`if derived_id != device.id: reject`	message-handler.ts:1137--1213
`if device.signedAt is stale: reject`	message-handler.ts:1137--1213
`if device.nonce is missing or mismatched: reject`	message-handler.ts:1137--1213
`payload_version = resolveDeviceSignaturePayloadVersion ↪ (...)`	message-handler.ts:1137--1213
`device_public_key = normalizeDevicePublicKeyBase64Url ↪ (...)`	message-handler.ts:1137--1213
`R1: device-identity checks are not applicable`	SourceDerived:hasDeviceIdentity=False;signaturepathisnotused
Input to the G2 main function	**Source**
`auth_decision = await resolveConnectAuthDecision(`	src/gateway/server/ws-connection/message-handler.ts:1215
`  state=connect_auth_state,`	message-handler.ts:1215--1252
`  hasDeviceIdentity=has_device_identity,`	message-handler.ts:1215--1252
`  deviceId=device.id,`	message-handler.ts:1215--1252

publicKey=device.publicKey,	message-handler.ts:1215--1252
role=role,	message-handler.ts:1215--1252
scopes=scopes,	message-handler.ts:1215--1252
rateLimiter=auth_rate_limiter,	message-handler.ts:1215--1252
clientIp=client_ip,	message-handler.ts:1215--1252
verifyBootstrapToken=verify_bootstrap_token,	message-handler.ts:1215--1252
verifyDeviceToken=verify_device_token	message-handler.ts:1215--1252
)	message-handler.ts:1215--1252

R1 input state	Runtime / upstream evidence
state.authOk = True	Observed:G0result="authorized"
state.authMethod = "token"	Observed:G0runtime
state.sharedAuthOk = True	Observed:G0runtime
state.sharedAuthProvided = True	Observed:G0runtime
state.bootstrapTokenCandidate = None (undefined)	SourceDerived:G0hasDeviceIdentity=False; auth-context.ts:138--140
state.deviceTokenCandidate = None (undefined)	SourceDerived:auth-context.ts:141--142 andobservedhasDeviceIdentity=False
state.deviceTokenCandidateSource = None (undefined)	SourceDerived:auth-context.ts: 141--142;candidateresolutionNotReached
hasDeviceIdentity = False	Observed:G0/G2runtime
deviceId = NOT APPLICABLE	SourceDerived:hasDeviceIdentity=False
publicKey = NOT APPLICABLE	SourceDerived:hasDeviceIdentity=False
role = "operator"	Observed:G2runtime
scopes = [	Observed:G2runtime
"operator.write"	Observed:G2runtime
]	Observed:G2runtime

Main G2 function	Source
async def resolveConnectAuthDecision(params):	src/gateway/server/ws-connection/auth-context.ts:188--207
if bootstrap verification does not require ↪ serialization:	auth-context.ts:188--207
return await resolveConnectAuthDecisionCore(params)	auth-context.ts:188--207
else:	auth-context.ts:188--207

```
      return await serialized bootstrap-token attempt
```
auth-context.ts:188--207

```
R1: bootstrapTokenCandidate = None
```
Observed:GOruntime

```
R1: direct core path is taken
```
SourceDerived:sourcecontrolflow

Step 2: Initialize final-auth state

```
auth_result = state.authResult
```
src/gateway/server/ws-connection/auth-context.ts:208--330

```
auth_ok = state.authOk
```
auth-context.ts:208--330

```
auth_method = state.authMethod
```
auth-context.ts:208--330

```
device_token_generation = None
```
auth-context.ts:208--330

```
R1: auth_ok = True
```
SourceDerived:upstreamGOvalue

```
R1: auth_method = "token"
```
SourceDerived:upstreamGOvalue

Step 3: Evaluate bootstrap-token path

```
if hasDeviceIdentity and deviceId and publicKey
```
src/gateway/server/ws-connection/auth-context.ts:208--330

```
   and state.bootstrapTokenCandidate:
```
auth-context.ts:208--330

```
      check bootstrap-token rate limit
```
auth-context.ts:208--330

```
      verify bootstrap token
```
auth-context.ts:208--330

```
      if valid:
```
auth-context.ts:208--330

```
         auth_ok = True
```
auth-context.ts:208--330

```
         auth_method = "bootstrap-token"
```
auth-context.ts:208--330

```
R1: bootstrapTokenCandidate = None
```
Observed

```
R1: bootstrap-token branch is not taken
```
SourceDerived:sourcecontrolflow

Step 4: Check whether device-token fallback is required

```
device_token = state.deviceTokenCandidate
```
src/gateway/server/ws-connection/auth-context.ts:208--330

```
if not hasDeviceIdentity
```
auth-context.ts:208--330

```
   or not deviceId
```
auth-context.ts:208--330

```
   or auth_ok
```
auth-context.ts:208--330

```
   or not device_token:
```
auth-context.ts:208--330

```
      return current decision
```
auth-context.ts:208--330

```
R1: hasDeviceIdentity = False
```
Observed

```
R1: auth_ok = True
```
Observed:GOruntime

```
R1: device token candidate = None (undefined)
```
SourceDerived:auth-context.ts:141--142;G0hasDeviceIdentity=False

```
R1: device-token verification is not reached
```
SourceDerived:sourcecontrolflow

Step 5: Apply device-token rate-limit guard

```
if rateLimiter exists:
```
src/gateway/server/ws-connection/auth-context.ts:208--330

```
   device_rate_check = rateLimiter.check(...)
```
auth-context.ts:208--330

```
   if not device_rate_check.allowed:
```
auth-context.ts:208--330

```
      return rate-limited auth failure
```
auth-context.ts:208--330

```
R1: rate-limit guard does not terminate the request
```
SourceDerived:shared-tokensuccess;device-tokenbranchnotreached

```
R1: exact limiter presence is not recorded
```
NotObserved:rationalefieldnotemitted

Step 6: Verify explicit device token

```
token_check = await verifyDeviceToken(
```
src/gateway/server/ws-connection/auth-context.ts:208--330

```
   deviceId=deviceId,
```
auth-context.ts:208--330

```
   token=device_token,
```
auth-context.ts:208--330

```
   role=role,
```
auth-context.ts:208--330

```
   scopes=scopes
```
auth-context.ts:208--330

```
)
```
auth-context.ts:208--330

```
if token_check.ok:
```
auth-context.ts:208--330

```
   auth_ok = True
```
auth-context.ts:208--330

```
   auth_method = "device-token"
```
auth-context.ts:208--330

```
R1: token_check.ok = NOT APPLICABLE
```
SourceDerived:device-tokenverificationbranchnotreached

```
R1: auth_ok = True
```
Observed:runtimevaluewithsourcecontext

```
R1: auth_method = "token"
```
Observed

Step 7: Return ConnectAuthDecision

Native G2 function output

```
return {
```
src/gateway/server/ws-connection/auth-context.ts:208--330

```
   "authResult": auth_result,
```
auth-context.ts

```
   "authOk": auth_ok,
```
auth-context.ts

```
   "authMethod": auth_method,
```
auth-context.ts

```
   "deviceTokenSharedGatewaySessionGeneration":
```
auth-context.ts

```
      device_token_generation
```
auth-context.ts

```
}
```
auth-context.ts

```
R1: authOk = True
```
Observed:runtimevaluewithsourcecontext

```
R1: authMethod = "token"
```
Observed

```
R1: device-token issuer generation = NOT RECORDED / not
↪ applicable
```
NotObserved:rationalefieldnotemitted

Step 8: Reject or accept the final authentication decision

```
authResult, authOk, authMethod = auth_decision
```
src/gateway/server/ws-connection/message-handler.ts:1215--1252

```
if not auth_ok:
```
message-handler.ts:1215--1252

```
   reject unauthorized connection
```
message-handler.ts:1215--1252

```
else:
```
message-handler.ts:1215--1252

```
   advance handshake to "auth_validated"
```
message-handler.ts:1215--1252

```
R1: auth_ok = True -> reject branch not taken
```
Observed

Step 9: Revalidate Gateway-auth generation

```
if session uses shared Gateway authentication:
```
src/gateway/server/ws-connection/message-handler.ts:1286--1295

```
   compare current and required auth generation
```
message-handler.ts:1286--1295

```
   if stale: close connection
```
message-handler.ts:1286--1295

```
R1: no generation-mismatch close occurs
```
SourceDerived:continuedhandshake

Step 10: Validate pairing and approved access

```
paired = await getPairedDevice(device.id)
```
src/gateway/server/ws-connection/message-handler.ts:1335--1757

```
validate device public key
```
message-handler.ts:1335--1757

```
validate requested role
```
message-handler.ts:1335--1757

```
validate requested scopes
```
message-handler.ts:1335--1757

```
if access requires upgrade: run pairing flow
```
message-handler.ts:1335--1757

```
R1: final role = "operator"
```
Observed

```
R1: final scopes = [operator.write]
```
Observed

```
R1: exact pairing sub-branch is not directly traced
```
NotObserved:rationalefieldnotemitted

Step 11: Device-token maintenance branch is not used by R1

```
if server-approved device baseline exists:
```
src/gateway/server/ws-connection/message-handler.ts:1759--1813

```
   deviceToken = await ensureDeviceToken(
```
message-handler.ts:1759--1813
```
      deviceId=device.id,
```
message-handler.ts:1759--1813
```
      role=role,
```
message-handler.ts:1759--1813
```
      scopes=scopes
```
message-handler.ts:1759--1813
```
   )
```
message-handler.ts:1759--1813

Step 12: Register authenticated Gateway client

```
next_client = GatewayWsClient(
```
src/gateway/server/ws-connection/message-handler.ts:2056--2065
```
   connect=connectParams,
```
message-handler.ts:2056--2065
```
   isDeviceTokenAuth=(authMethod == "device-token"),
```
message-handler.ts:2056--2065
```
   ...
```
message-handler.ts:2056--2065
```
)
```
message-handler.ts:2056--2065
```
setClient(next_client)
```
message-handler.ts:2056--2065
```
set handshake state = "connected"
```
message-handler.ts:2056--2065
```
R1: isDeviceTokenAuth = False
```
SourceDerived:shared-tokenpath;device-tokenfallbacknotreached

Step 13: Construct authenticated hello-ok payload

```
hello_ok.auth.role = role
```
src/gateway/server/ws-connection/message-handler.ts:2169--2204
```
hello_ok.auth.scopes = authenticated_scopes
```
message-handler.ts:2169--2204
```
hello_ok.auth may include deviceToken
```
message-handler.ts:2169--2204
```
R1: role = "operator"
```
Observed
```
R1: scopes = [operator.write]
```
Observed

Step 14: Send hello-ok and finalize the connection

```
await sendFrame(hello_ok)
```
src/gateway/server/ws-connection/message-handler.ts:2237--2273
```
emit gateway authentication success event
```
message-handler.ts:2237--2273
```
advance handshake phase = "ready"
```
message-handler.ts:2237--2273
```
R1: stage = "G2"
```
Observed
```
R1: event = "connection_identity_finalized"
```
Observed
```
R1: result = "pass"
```
Observed
```
R1: authMethod = "token"
```
Observed
```
R1: role = "operator"
```
Observed

R1: clientMode = "webchat"

Observed



G3: Method Authorization

Purpose: This step performs method-level authorization after the Gateway connection has been authenticated. It determines whether the authenticated client may invoke `chat.send` from its connection role and scopes. It does not validate the `chat.send` request body or inspect

the semantic content of the user message. In R1, the authenticated client has role `operator` and includes the `operator.write` scope. The administrator shortcut is not taken because `operator.admin` is absent, so the ordinary `operator.write` required-scope check is reached.

Python-style pseudocode	**Exact source**
`# Gateway request-dispatch call site`	src/gateway/server-methods.ts: 734--738

`auth_error = authorizeGatewayMethod(`
`  req.method, client, req.params, method_registry`
`)`
	src/gateway/server-methods.ts:734

`if auth_error is not None:`	src/gateway/server-methods.ts: 735--738
`  respond(False, None, auth_error)`	src/gateway/server-methods.ts:736
`  return`	src/gateway/server-methods.ts: 735--738
`# authorizeGatewayMethod implementation`	src/gateway/server-methods.ts: 262--299

`def authorize_gateway_method(`
`  method, client, params, method_registry`
`):`
	src/gateway/server-methods.ts: 262--267

`if client.connect is None:`	src/gateway/server-methods.ts: 270--272
`  return None`	src/gateway/server-methods.ts:271
`if method == "health":`	src/gateway/server-methods.ts: 273--275
`  return None`	src/gateway/server-methods.ts:274
`role_raw = client.connect.role or "operator"`	src/gateway/server-methods.ts:276
`role = parseGatewayRole(role_raw)`	src/gateway/server-methods.ts:277
`if role is None:`	src/gateway/server-methods.ts: 278--280
`   return invalid_request( "unauthorized role" )`	src/gateway/server-methods.ts:279
`scopes = client.connect.scopes or []`	src/gateway/server-methods.ts:281

`if not isRoleAuthorizedForMethod(`
`  role, method`
`):`
	src/gateway/server-methods.ts: 282--284

`   return invalid_request( "unauthorized role" )`	src/gateway/server-methods.ts:283
`# Helper semantics: node RPCs require role="node";` `# all other RPCs require role="operator".`	src/gateway/role-policy.ts:24--29
`if role == "node":`	src/gateway/server-methods.ts: 285--287

Python-style pseudocode | **Exact source**

```
   return None
```
src/gateway/server-methods.ts:286

```
admin_scope_present =
    "operator.admin" in scopes
```
DERIVED FROM: src/gateway/server-methods.ts:288--290 src/gateway/operator-scopes.ts:3

```
if admin_scope_present:
```
src/gateway/server-methods.ts:288--290

```
   return None
```
src/gateway/server-methods.ts:289

```
registered_scope =
    method_registry.getScope(method)
```
src/gateway/server-methods.ts:291

```
# For chat.send, the registry value is
# "operator.write".
```
src/gateway/methods/core-descriptors.ts:231

```
if isOperatorScope( registered_scope ):
```
src/gateway/server-methods.ts:292--294

```
   scope_auth =
       authorizeOperatorScopesForRequiredScope(
          registered_scope, scopes
       )
```
src/gateway/server-methods.ts:292--293

```
else:
```
src/gateway/server-methods.ts:292--294

```
   scope_auth =
       authorizeOperatorScopesForMethod(
          method, scopes, params
       )
```
src/gateway/server-methods.ts:292--294

```
if not scope_auth.allowed:
```
src/gateway/server-methods.ts:295--297

```
    return invalid_request( "missing scope" )
```
src/gateway/server-methods.ts:296

```
return None
```
src/gateway/server-methods.ts:298

```
# If the required-scope branch is reached for chat.send,
# registered_scope = "operator.write", and the static helper
# allows the call when "operator.write" is present in scopes.
```
SOURCE SHAPE: src/gateway/methods/core-descriptors.ts:231 src/gateway/method-scopes.ts:257--274 src/gateway/method-scopes.ts:270--271

R1 concrete execution | **Observed / derived state**

```
method = "chat.send"
```
Observed: G3 input

```
client.connect is present = True
```
SourceDerived: G3 follows the authenticated-client path

```
method == "health" = False
```
SourceDerived: method is "chat.send"

```
role = "operator"
```
Observed: G3 input

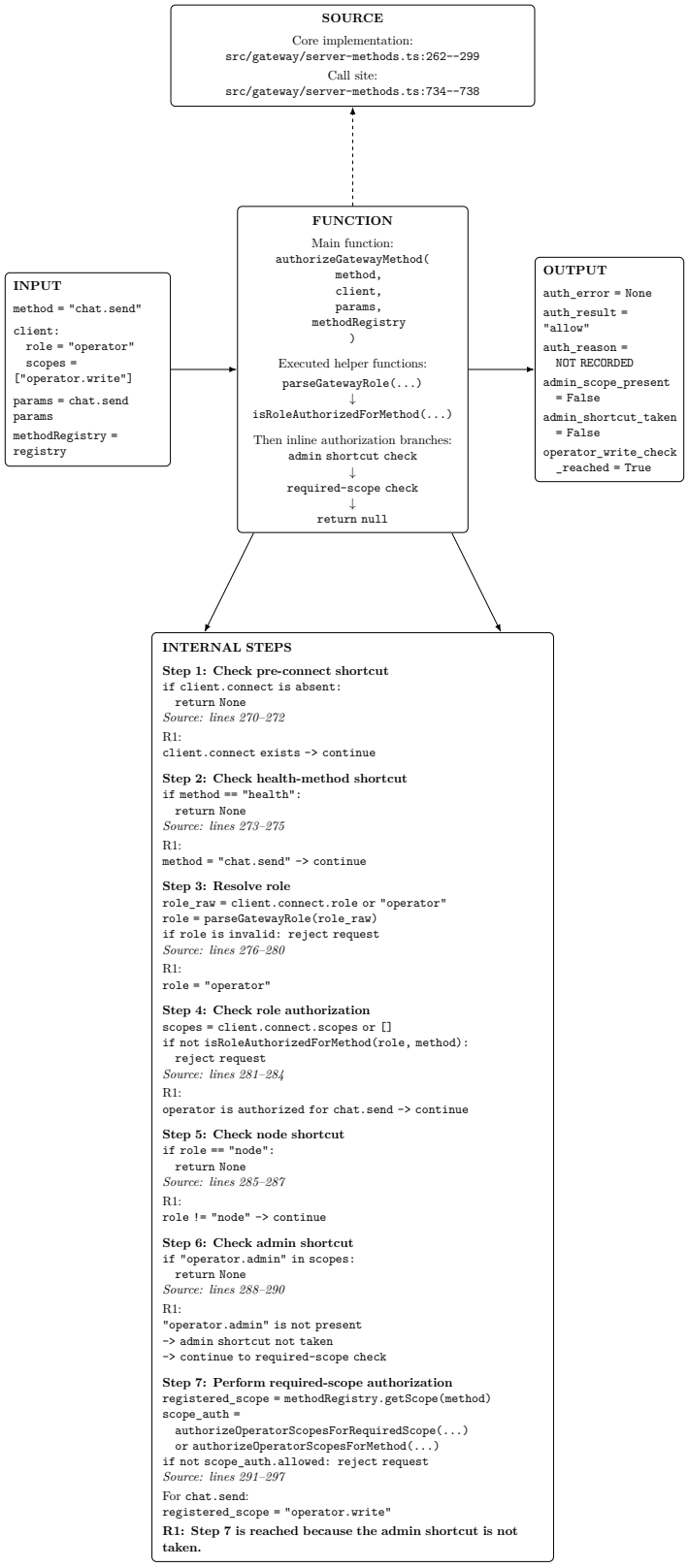
Python-style pseudocode	Exact source

`scopes = [`
`  "operator.write"`
`]`
	Observed: G3 input

`role_authorization = pass`	SourceDerived: chat.send is an operator RPC and role="operator"
`node_shortcut_taken = False`	SourceDerived: role != "node"
`admin_scope_present = False`	SourceDerived: "operator.admin" is not present in the observed scopes
`admin_shortcut_taken = False`	SourceDerived: condition at lines 288--290 is false
`operator_write_check_reached = True`	SourceDerived: execution continues past line 290
`registered_scope_lookup_reached = True`	SourceDerived: method_registry.getScope(method) at line 291 is reached
`auth_error = None`	SourceDerived: authorizeGatewayMethod returns null on the ordinary operator.write check
`auth_result = "allow"`	Observed: G3 TraceClaw event
`auth_reason = NOT RECORDED`	NotObserved: R1 G3 runtime event has result only

G3: Method Authorization Flow Diagram

Purpose: This step performs method-level authorization for the authenticated Gateway client. It checks whether the current operator is allowed to invoke `chat.send`. In R1, the authenticated connection contains `operator.write`; the administrator shortcut is not taken, so the normal `operator.write` scope-checking branch is reached.



G4: `chat.send` Request Validation

Purpose: This step validates the incoming `chat.send` RPC envelope before its contents are processed further. It validates the request against `ChatSendParamsSchema`, resolves reconnect-resume metadata, normalizes any explicit originating route, and rejects protected provenance or routing fields when the caller lacks administrator scope.

Python-style pseudocode	Exact source

```
def chat_send(params, client, context):
```
src/gateway/server-methods/chat.ts:3699

```
client_info = client.connect.client
```
src/gateway/server-methods/chat.ts:3701

```
resume = resolveControlUiReconnectResumeParams(params,
↪ client_info)
```
src/gateway/server-methods/chat.ts:3702

```
## helper implementation
```
src/gateway/server-methods/chat.ts:341--357

```
schema_valid = validateChatSendParams(resume.params)
```
src/gateway/server-methods/chat.ts:3703

```
if not schema_valid:
```
src/gateway/server-methods/chat.ts:3703--3713

```
return invalid_chat_send_params
```
src/gateway/server-methods/chat.ts:3704--3712

```
p = resume.params
```
src/gateway/server-methods/chat.ts:3714--3739

```
suppress_command = (p.suppressCommandInterpretation is True)
```
src/gateway/server-methods/chat.ts:3740

```
explicit_origin = normalizeExplicitChatSendOrigin(...)
```
src/gateway/server-methods/chat.ts:3741--3749

```
if not explicit_origin.ok:
```
src/gateway/server-methods/chat.ts:3747--3749

```
return invalid_origin_error
```
src/gateway/server-methods/chat.ts:3747--3749

```
has_explicit_origin = (explicit_origin.value is not None)
```
SourceDerived:chat.ts:3741--3749

```
has_privileged_fields = bool(p.systemInputProvenance or
```
src/gateway/server-methods/chat.ts:3751--3755

```
p.systemProvenanceReceipt or
```
src/gateway/server-methods/chat.ts:3751--3755

```
suppress_command)
```
src/gateway/server-methods/chat.ts:3751--3755

```
if (has_privileged_fields or has_explicit_origin)
```
src/gateway/server-methods/chat.ts:3751--3756

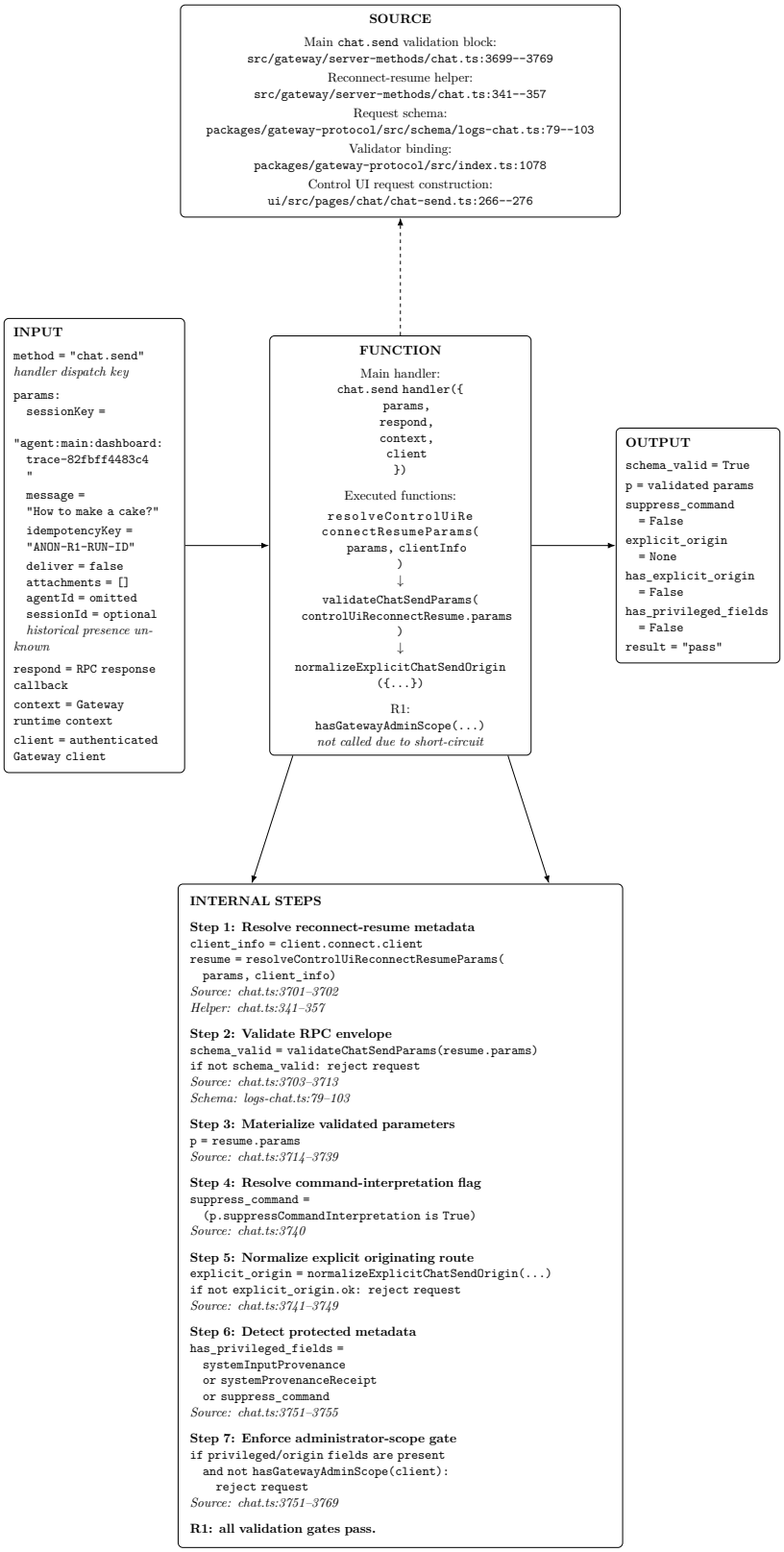
```
and not hasGatewayAdminScope(client):
```
src/gateway/server-methods/chat.ts:3756

`return admin_scope_required`	src/gateway/server-methods/chat.ts:3757--3769
`ChatSendParamsSchema = sessionKey, agentId?, sessionId?, ↪ message, ...`	packages/gateway-protocol/src/schema/logs-chat.ts:79--103
`validateChatSendParams = validator(ChatSendParamsSchema)`	packages/gateway-protocol/src/index.ts:1078
`ControlUI.request("chat.send", rpc_params)`	ui/src/pages/chat/chat-send.ts:266--276
`rpc_params["sessionKey"] = routing.sessionKey`	ui/src/pages/chat/chat-send.ts:266--276
`if routing.sessionId: rpc_params["sessionId"] = routing. ↪ sessionId`	ui/src/pages/chat/chat-send.ts:266--276
`rpc_params["message"] = params.message`	ui/src/pages/chat/chat-send.ts:266--276
`rpc_params["deliver"] = False`	ui/src/pages/chat/chat-send.ts:266--276
`rpc_params["idempotencyKey"] = params.runId`	ui/src/pages/chat/chat-send.ts:266--276
`rpc_params["attachments"] = buildApiAttachments(params. ↪ attachments)`	ui/src/pages/chat/chat-send.ts:266--276
`R1: schema_valid = True`	SourceDerived: successfulvalidationpath
`R1: has_privileged_fields = False`	Observed:G4TraceClawevent
`R1: has_explicit_origin = False`	Observed:G4TraceClawevent
`R1: suppress_command = False`	SourceDerived:chat.ts:3740
`R1: result = "pass"`	Observed:G4TraceClawevent

Strict note: `sessionId` is optional in the RPC schema and is conditionally included by the Control UI. The R1 G4 RPC payload does not archive a Gateway Session identifier, and this appendix does not fill it from a different layer. Therefore this table does not claim that the field was unconditionally present in the original G4 RPC payload.

G4: `chat.send` Request Validation Flow Diagram

Purpose: This step validates the incoming `chat.send` RPC envelope before its contents are processed further. It validates the request against `ChatSendParamsSchema`, resolves Control UI reconnect-resume metadata, normalizes any explicitly supplied originating route, and rejects protected provenance or routing fields when the caller lacks administrator scope. For R1, the request conforms to the schema, contains no privileged metadata or explicit origin, and therefore passes G4 without rejection.



Strict note: `sessionId` is optional in `ChatSendParamsSchema` and is conditionally included by the Control UI. The R1 G4 payload does not record a Gateway `sessionId`, and the later AR-layer Session ID must not be used to fill that Gateway-stage field. Therefore, G4 marks `sessionId` as `NOT RECORDED`.

G5: Message Sanitization and Classification

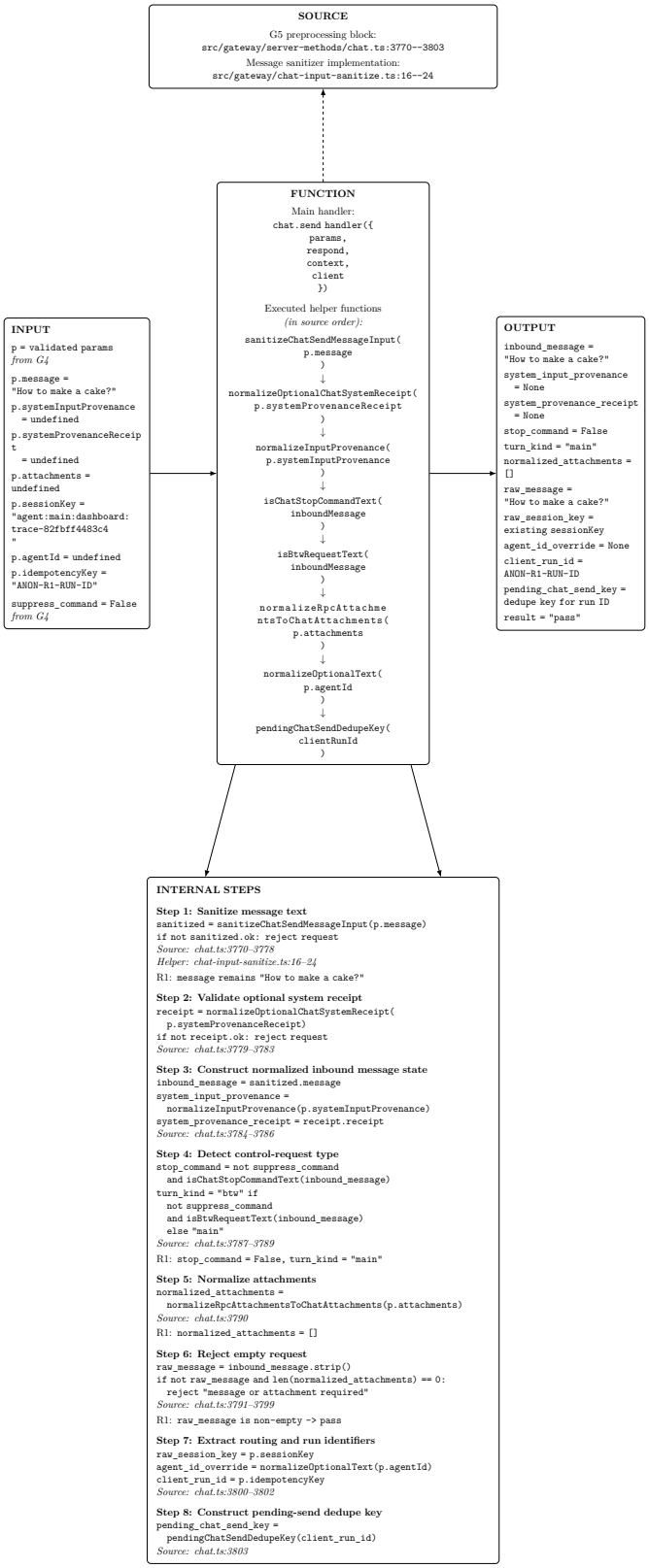
Purpose: This step normalizes the validated message, rejects invalid message content, identifies control requests such as `/stop` or `/btw`, normalizes attachments, and extracts the session key, Agent override, and per-request run identifier needed by the following routing stages.

Python-style pseudocode	Exact source
`sanitized = sanitizeChatSendMessageInput(p.message)`	src/gateway/server-methods/chat.ts:3770
`## sanitizeChatSendMessageInput implementation`	src/gateway/chat-input-sanitize.ts:16--24
`if not sanitized.ok:`	src/gateway/server-methods/chat.ts:3771--3778
`return sanitized.error`	src/gateway/server-methods/chat.ts:3772--3777
`receipt = normalizeOptionalChatSystemReceipt(p.systemProvenanceReceipt)`	src/gateway/server-methods/chat.ts:3779
`if not receipt.ok:`	src/gateway/server-methods/chat.ts:3780--3783
`return receipt.error`	src/gateway/server-methods/chat.ts:3781--3783
`inbound_message = sanitized.message`	src/gateway/server-methods/chat.ts:3784
`system_input_provenance = normalizeInputProvenance(p.systemInputProvenance)`	src/gateway/server-methods/chat.ts:3785
`system_provenance_receipt = receipt.receipt`	src/gateway/server-methods/chat.ts:3786
`stop_command = not suppress_command and isChatStopCommandText(inbound_message)`	src/gateway/server-methods/chat.ts:3787
`turn_kind = "btw" if isBtwRequestText(inbound_message) else "main"`	src/gateway/server-methods/chat.ts:3788--3789
`attachments = normalizeRpcAttachmentsToChatAttachments(p.attachments)`	src/gateway/server-methods/chat.ts:3790
`raw_message = inbound_message.strip()`	src/gateway/server-methods/chat.ts:3791
`if not raw_message and len(attachments) == 0:`	src/gateway/server-methods/chat.ts:3792--3799
`return "message or attachment required"`	src/gateway/server-methods/chat.ts:3793--3798

`raw_session_key = p.sessionKey`	src/gateway/server-methods/chat.ts:3800
`agent_id_override = normalizeOptionalText(p.agentId)`	src/gateway/server-methods/chat.ts:3801
`client_run_id = p.idempotencyKey`	src/gateway/server-methods/chat.ts:3802
`pending_chat_send_key = pendingChatSendDedupeKey(client_run_id)`	src/gateway/server-methods/chat.ts:3803
`R1: inbound_message = "How to make a cake?"`	Observed: savedmessage;sourceatchat.ts:3784
`R1: message_length = 19`	Observed:G5TraceClawevent
`R1: message_changed_by_sanitization = False`	Observed:G5TraceClawevent
`R1: stop_command = False`	SourceDerived:chat.ts:3787
`R1: turn_kind = "main"`	SourceDerived:chat.ts:3788--3789
`R1: attachments = []`	SourceDerived:chat.ts:3790
`R1: raw_session_key = SESSION_KEY`	chat.ts:3800
`R1: client_run_id = RUN_ID`	chat.ts:3802

G5: Message Sanitization and Classification Flow Diagram

Purpose: This step converts the validated `chat.send` request into the normalized message representation used by the subsequent routing stages. It sanitizes the message text, validates optional system provenance metadata, detects control requests such as `/stop` and `/btw`, normalizes attachments, rejects an empty message with no attachments, and extracts the existing Session key, optional Agent override, and per-request run identifier. For R1, the message `"How to make a cake?"` is unchanged by sanitization, is classified as a normal `main` turn, contains no attachments, and proceeds with the existing Session key and run ID.



G6: Requested Agent Resolution

Purpose: This step determines whether the current request itself explicitly selects an Agent. An explicit `agentId` takes precedence. If no explicit Agent is supplied, the session key is parsed, but the parsed Agent is returned at this stage only when the session resolves to the special `global` store. For R1, the Dashboard session is not a global-store request, so `requestedAgentId` remains undefined.

Python-style pseudocode	Exact source
`requested_agent_id = resolveRequestedChatAgentId(...)`	src/gateway/server-methods/chat.ts:3804--3808
`def resolveRequestedChatAgentId(params):`	src/gateway/server-methods/chat.ts:725--745
`explicit_agent_id = normalizeOptionalText(params.agentId)`	src/gateway/server-methods/chat.ts:730
`if explicit_agent_id:`	src/gateway/server-methods/chat.ts:731--733
`return normalizeAgentId(explicit_agent_id)`	src/gateway/server-methods/chat.ts:732
`if params.cfg is None:`	src/gateway/server-methods/chat.ts:734--736
`return None`	src/gateway/server-methods/chat.ts:734--736
`parsed = parseAgentSessionKey(params.requestedSessionKey.strip ↪ ())`	src/gateway/server-methods/chat.ts:737
`if parsed is None or parsed.agentId is None:`	src/gateway/server-methods/chat.ts:738--742
`return None`	src/gateway/server-methods/chat.ts:738--742
`store_key = resolveSessionStoreKey(cfg=params.cfg,`	src/gateway/server-methods/chat.ts:739--741
`sessionKey=params.requestedSessionKey)`	src/gateway/server-methods/chat.ts:739--741
`if store_key != "global":`	src/gateway/server-methods/chat.ts:738--742
`return None`	src/gateway/server-methods/chat.ts:738--742
`return normalizeAgentId(parsed.agentId)`	src/gateway/server-methods/chat.ts:744
`session_load_options = dict(agentId=requested_agent_id)`	src/gateway/server-methods/chat.ts:3809
`if requested_agent_id is not None else None`	src/gateway/server-methods/chat.ts:3809
R1: explicit_agent_id = None	SourceDerived:chat.ts:730--733

```
R1: parsed.agentId = "main"
```

SourceDerived: parsercallatchat.ts:737

```
R1: parsed.rest = "dashboard:trace-82fbff4483c4"
```

SourceDerived: parsercallatchat.ts:737

```
R1: store_key != "global"
```

SourceDerived:chat.ts:738--742

```
R1: requested_agent_id = None
```

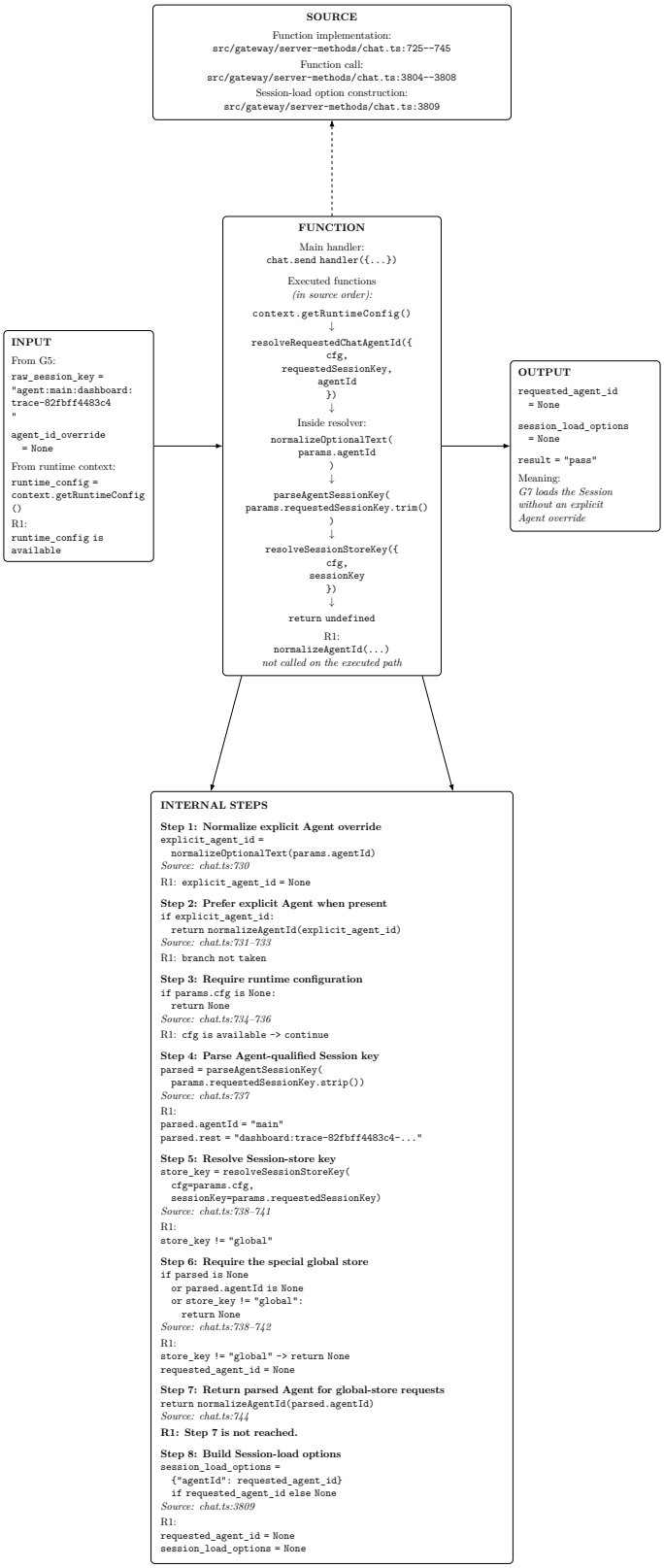
chat.ts:738--742

```
R1: session_load_options = None
```

chat.ts:3809

G6: Requested Agent Resolution Flow Diagram

Purpose: This step determines whether the current request explicitly selects an Agent for the subsequent Session lookup. An explicit `agentId` takes precedence. If no explicit Agent is supplied, the Session key is parsed and its embedded Agent identifier is considered only when the request resolves to the special `global` Session store. For R1, no explicit Agent override is supplied. Although the Dashboard Session key contains the Agent identifier `main`, it does not resolve to the `global` store at this stage. Therefore, `requestedAgentId` remains undefined and no Agent-specific Session-load option is created.



G7: Session Resolution and Store Lookup

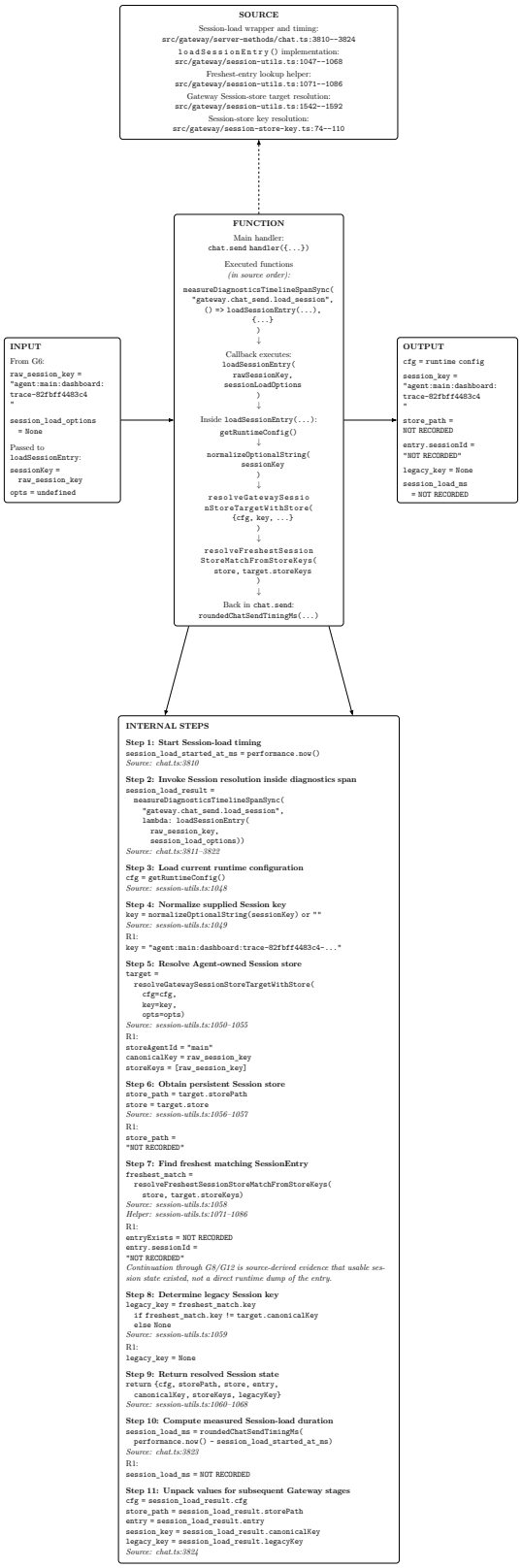
Purpose: This step resolves the supplied session key. The Gateway measures the session-resolution operation, invokes `loadSessionEntry()`, resolves the appropriate session store, searches the candidate store keys, and returns the canonical key plus any loaded session record. R1 directly records the canonical Session key and the G7 `resolved` outcome. It does not directly record the store path, the loaded `entry` object, or `entry.sessionId`.

Python-style pseudocode	Exact source
`session_load_started_at_ms = performance.now()`	src/gateway/server-methods/chat.ts:3810
`session_load_result = measureDiagnosticsTimelineSpanSync(`	src/gateway/server-methods/chat.ts:3811--3822
`"gateway.chat_send.load_session",`	src/gateway/server-methods/chat.ts:3812
`lambda: loadSessionEntry(raw_session_key, session_load_options) ↪ ,`	src/gateway/server-methods/chat.ts:3813
`phase="agent-turn",`	src/gateway/server-methods/chat.ts:3814--3821
`)`	src/gateway/server-methods/chat.ts:3811--3822
`session_load_ms = roundedChatSendTimingMs(`	src/gateway/server-methods/chat.ts:3823
`performance.now() - session_load_started_at_ms)`	src/gateway/server-methods/chat.ts:3823
`cfg = session_load_result.cfg`	src/gateway/server-methods/chat.ts:3824
`store_path = session_load_result.storePath`	src/gateway/server-methods/chat.ts:3824
`entry = session_load_result.entry`	src/gateway/server-methods/chat.ts:3824
`session_key = session_load_result.canonicalKey`	src/gateway/server-methods/chat.ts:3824
`legacy_key = session_load_result.legacyKey`	src/gateway/server-methods/chat.ts:3824
`def loadSessionEntry(session_key, opts=None):`	src/gateway/session-utils.ts:1047--1068
`cfg = getRuntimeConfig()`	src/gateway/session-utils.ts:1048
`key = normalizeOptionalString(session_key) or ""`	src/gateway/session-utils.ts:1049
`target = resolveGatewaySessionStoreTargetWithStore(...)`	src/gateway/session-utils.ts:1050--1055
`store_path = target.storePath`	src/gateway/session-utils.ts:1056
`store = target.store`	src/gateway/session-utils.ts:1057
`freshest_match = resolveFreshestSessionStoreMatchFromStoreKeys(`	src/gateway/session-utils.ts:1058

store, target.storeKeys)	src/gateway/session-utils.ts:1058
## helper scans candidate keys and selects newest updatedAt	src/gateway/session-utils.ts: 1071--1086
legacy_key = freshest_match.key	src/gateway/session-utils.ts:1059
if freshest_match.key != target.canonicalKey **else** None	src/gateway/session-utils.ts:1059
return dict(	src/gateway/session-utils.ts: 1060--1068
cfg=cfg,	src/gateway/session-utils.ts: 1060--1068
storePath=store_path,	src/gateway/session-utils.ts:1062
store=store,	src/gateway/session-utils.ts:1063
entry=freshest_match.entry **if** freshest_match **else** None,	src/gateway/session-utils.ts:1064
canonicalKey=target.canonicalKey,	src/gateway/session-utils.ts:1065
storeKeys=target.storeKeys,	src/gateway/session-utils.ts:1066
legacyKey=legacy_key)	src/gateway/session-utils.ts: 1067--1068
R1: canonical_session_key = SESSION_KEY	Observed;session-utils.ts:1065
R1: store_agent_id = "main"	SourceDerived:session-utils.ts: 1542--1592
R1: store_path = NOT RECORDED	NotObserved:absentfromR1trace;so urcecomputestarget.storePath
R1: store_keys = [SESSION_KEY]	Observed;session-utils.ts:1066
R1: entry_exists = NOT RECORDED	NotObserved: usablesessionstateabsent;SourceD erivedfromcontinuedcontrolflow
R1: session_id = entry.sessionId	SourceDerived: readsite;concretevaluenotemitted
R1: session_id = "NOT RECORDED"	NotObserved: absentfromGatewaytrace
backing_session_id = entry.sessionId **or** requested_session_id	src/gateway/server-methods/chat. ts:3848--3849
R1: legacy_key = None	session-utils.ts:1059,1067
R1: session_load_ms = NOT RECORDED	NotObserved:absentfromR1trace;ti mingsourceexistsincode

G7: Session Resolution and Store Lookup Flow Diagram

Purpose: This step resolves the supplied Session key. The Gateway invokes `loadSessio nEntry()` inside a diagnostics and timing wrapper. The function obtains the current runtime configuration, canonicalizes the supplied key, resolves the appropriate Agent-owned Session store, and searches candidate store keys. For R1, the trace records that G7 resolves and that the canonical key remains unchanged. It does not directly record the store path, the loaded `entry` object, `entry.sessionId`, or the measured Session-load duration.



G8: Agent / Session Consistency Validation

Purpose:

This step validates whether an explicitly requested Agent is compatible with the current Session routing, computes a backing Session identity when one is available, and rejects Sessions whose owning Agent has been removed from the current configuration. For R1, no explicit Agent override was supplied, so Agent-selection validation succeeds immediately. The trace records the G8 `pass` outcome. It does not directly emit the loaded `SessionEntry` or the concrete backing Session ID, so those fields remain `NOT RECORDED`.

Python-style pseudocode	Exact source / evidence
`selected_agent = validateChatSelectedAgent(`	src/gateway/server-methods/chat.ts: 3839--3847
`  cfg=cfg,`	src/gateway/server-methods/chat.ts: 3839--3843
`  requestedSessionKey=raw_session_key,`	src/gateway/server-methods/chat.ts:3841
`  agentId=requested_agent_id,`	src/gateway/server-methods/chat.ts:3842
`)`	src/gateway/server-methods/chat.ts: 3839--3843
`if not selected_agent.ok:`	src/gateway/server-methods/chat.ts: 3844--3847
`  return invalid_request(selected_agent.error)`	src/gateway/server-methods/chat.ts: 3845--3846
`def validateChatSelectedAgent(params):`	src/gateway/server-methods/chat.ts: 690--723
`  agent_id = normalizeAgentId(params.agentId) if params.agentId else None`	src/gateway/server-methods/chat.ts:695
`  if agent_id is None:`	src/gateway/server-methods/chat.ts: 696--698
`    return {"ok": True}`	src/gateway/server-methods/chat.ts:697
`  if agent_id not in listAgentIds(params.cfg):`	src/gateway/server-methods/chat.ts: 699--701
`    return {"ok": False, "error": "Unknown agent id"}`	src/gateway/server-methods/chat.ts:700
`  requested_session_key = params.requestedSessionKey.strip()`	src/gateway/server-methods/chat.ts:702
`  parsed = parseAgentSessionKey(requested_session_key)`	src/gateway/server-methods/chat.ts:703
`  if parsed and normalizeAgentId(parsed.agentId) != agent_id:`	src/gateway/server-methods/chat.ts:704
`    return {"ok": False, "error": "agentId/sessionKey ↪ mismatch"}`	src/gateway/server-methods/chat.ts: 705--708
`  if requested_session_key.lower() == "global":`	src/gateway/server-methods/chat.ts:710
`    return {"ok": True, "agentId": agent_id}`	src/gateway/server-methods/chat.ts:711

```
   if resolveSessionStoreKey(cfg=params.cfg,
sessionKey=requested_session_key) == "global":
      return {"ok": True, "agentId": agent_id}

   if parsed is None or
normalizeAgentId(parsed.agentId) != agent_id:
      return {"ok": False, "error": "agentId/sessionKey
↪ mismatch"}

   return {"ok": True, "agentId": agent_id}
```

src/gateway/server-methods/chat.ts:713

src/gateway/server-methods/chat.ts:714

src/gateway/server-methods/chat.ts:716

src/gateway/server-methods/chat.ts:718--720

src/gateway/server-methods/chat.ts:722

```
requested_session_id =
normalizeOptionalText(p.sessionId)
backing_session_id = entry.sessionId if entry is
not None else requested_session_id
```

src/gateway/server-methods/chat.ts:3848

src/gateway/server-methods/chat.ts:3849

```
deleted_agent_id =
resolveDeletedAgentIdFromSessionKey(
   cfg,
   session_key,
   entry,
   acpMetadataSessionKey=(legacy_key or
session_key),
)


if deleted_agent_id is not None:


   return invalid_request("Agent no longer exists
in configuration")
```

src/gateway/server-methods/chat.ts:3850--3852

src/gateway/server-methods/chat.ts:3850

src/gateway/server-methods/chat.ts:3850

src/gateway/server-methods/chat.ts:3851

src/gateway/server-methods/chat.ts:3850--3852

src/gateway/server-methods/chat.ts:3853--3863

src/gateway/server-methods/chat.ts:3854--3861

```
R1: requested_agent_id = None

R1: agent_id = None


R1: selected_agent = {"ok": True}


R1: later mismatch checks are not reached

R1: requested_session_id = unknown


R1: entry.sessionId =

   "NOT RECORDED"

R1: backing_session_id =


   "NOT RECORDED"
```

Observed:outputcarriedfromG6

SourceDerived:
src/gateway/server-methods/chat.ts:695

SourceDerived:src/gateway/server-methods
/chat.ts:696--698

SourceDerived:earlyreturnatchat.ts:697

NotObserved:Observabilitygap;originalraw
RPCframenotarchived

NotObserved:absentfromR1Gatewaytrace

NotObserved:absentfromR1trace

SourceDerived:
readsite;concretevaluenotemitted

NotObserved:
donotsubstituteAR-layersessionId

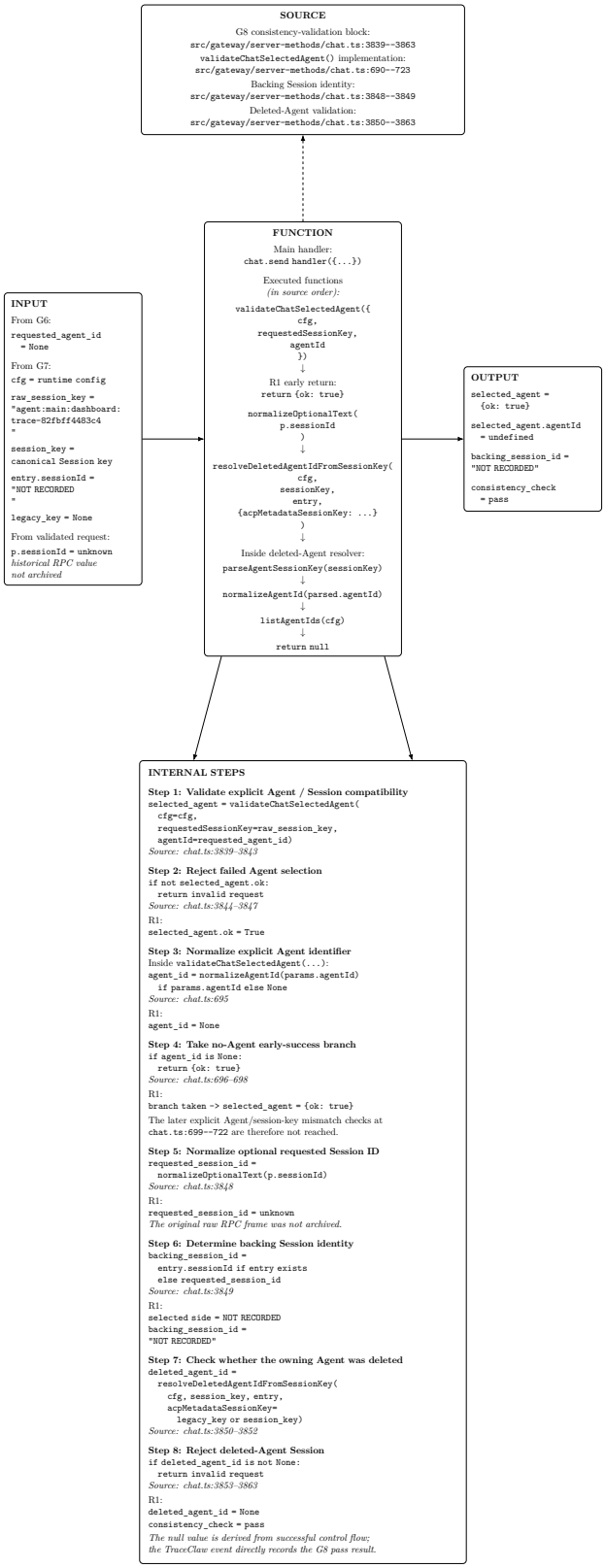
```
R1: deleted_agent_id = None
R1: result = "pass"
```

SourceDerived:chat.ts:3853--3863

Observed:G8TraceClawevent

G8: Agent / Session Consistency Validation Flow Diagram

Purpose: This step validates whether an explicitly requested Agent is compatible with the current Session routing, computes a backing Session identity when one is available, and rejects Sessions whose owning Agent has been removed from the current configuration. For R1, no explicit Agent override was supplied, so Agent-selection validation succeeds immediately. The concrete backing Session ID is not emitted in the Gateway trace. Therefore, G8 passes and execution proceeds to G9, while the backing-identity details remain a rationale-level gap.



Strict note: The exact historical R1 value of `p.sessionId` is not independently observable because the original raw RPC frame was not archived. G7 also does not emit the loaded `entry` object or `entry.sessionId`. The source at `chat.ts:3849` shows where a backing identity would be selected, but R1 does not record which operand supplied it. The R1 `backingSessionId` is therefore `NOT RECORDED`; the AR-layer Session ID must not be used to fill this Gateway field.

G9: Effective Agent Resolution

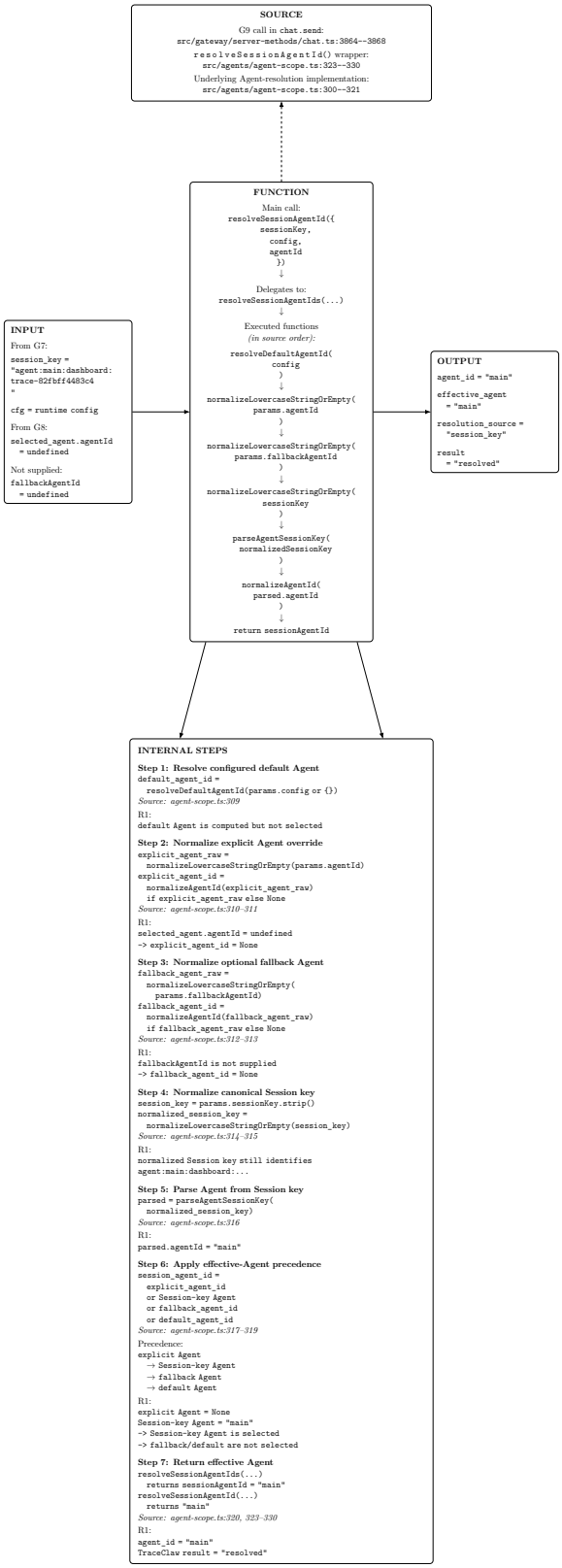
Purpose: This step resolves the effective Gateway-side Agent for the current Session after the Agent / Session consistency checks in G8 have passed. The Gateway invokes `resolveSessionAgentId()` with the canonical Session key, current runtime configuration, and any Agent identifier validated by G8. The underlying resolver applies the precedence explicit Agent → Session-key Agent → fallback Agent → default Agent. For R1, no explicit Agent survives G8, while the canonical Session key encodes `main`. Therefore, the Session-key Agent branch determines the effective Agent as `main`. The semantic content of `"How to make a cake?"` is not used for this routing decision.

Python-style pseudocode	Exact source / evidence
`agent_id = resolveSessionAgentId(`	src/gateway/server-methods/chat.ts: 3864--3868
`  sessionKey=session_key,`	src/gateway/server-methods/chat.ts:3865
`  config=cfg,`	src/gateway/server-methods/chat.ts:3866
`  agentId=selected_agent.agentId,`	src/gateway/server-methods/chat.ts:3867
`)`	src/gateway/server-methods/chat.ts: 3864--3868
`def resolveSessionAgentId(params):`	src/agents/agent-scope.ts:323--330
`  return resolveSessionAgentIds(params).sessionAgentId`	src/agents/agent-scope.ts:329
`def resolveSessionAgentIds(params):`	src/agents/agent-scope.ts:300--321
`  default_agent_id = resolveDefaultAgentId(params.config ↪ or {})`	src/agents/agent-scope.ts:309
`  explicit_agent_raw = normalizeLowercaseStringOrEmpty( ↪ params.agentId)`	src/agents/agent-scope.ts:310
`  explicit_agent_id = normalizeAgentId( ↪ explicit_agent_raw)`	src/agents/agent-scope.ts:311
`    if explicit_agent_raw else None`	src/agents/agent-scope.ts:311
`  fallback_agent_raw = normalizeLowercaseStringOrEmpty( ↪ params.fallbackAgentId)`	src/agents/agent-scope.ts:312
`  fallback_agent_id = normalizeAgentId( ↪ fallback_agent_raw)`	src/agents/agent-scope.ts:313
`    if fallback_agent_raw else None`	src/agents/agent-scope.ts:313
`  session_key = params.sessionKey.strip() if params. ↪ sessionKey else None`	src/agents/agent-scope.ts:314
`  normalized_session_key = ↪ normalizeLowercaseStringOrEmpty(session_key)`	src/agents/agent-scope.ts:315
`    if session_key else None`	src/agents/agent-scope.ts:315
`  parsed = parseAgentSessionKey(normalized_session_key)`	src/agents/agent-scope.ts:316
`    if normalized_session_key else None`	src/agents/agent-scope.ts:316
`  session_agent_id = explicit_agent_id or`	src/agents/agent-scope.ts:317--319
`    normalizeAgentId(parsed.agentId) if parsed and ↪ parsed.agentId`	src/agents/agent-scope.ts:317--319

`else fallback_agent_id or default_agent_id`	src/agents/agent-scope.ts:317--319
`return {"defaultAgentId": default_agent_id,`	src/agents/agent-scope.ts:320
`"sessionAgentId": session_agent_id}`	src/agents/agent-scope.ts:320
`R1: session_key =`	Observed: canonicalSessionkeycarriedfromG7
`"agent:main:dashboard:trace-82fbff4483c4"`	Observed:G7canonicalSessionkey
`R1: selected_agent.agentId = None`	SourceDerived:G8returns\{ok: true\}withoutagentId
`R1: explicit_agent_id = None`	SourceDerived:agent-scope.ts:310--311
`R1: fallback_agent_id = None`	SourceDerived:fallbackAgentIdisnotsuppliedatchat.ts:3864--3868
`R1: parsed.agentId = "main"`	SourceDerived:agent-scope.ts: 314--316+canonicalSessionkey
`R1: Session-key Agent branch is selected`	SourceDerived:agent-scope.ts:317--319
`R1: fallback Agent branch is not selected`	SourceDerived:agent-scope.ts:317--319
`R1: default Agent branch is not selected`	SourceDerived:agent-scope.ts:317--319
`R1: agent_id = "main"`	Observed:G9TraceClawevent
`R1: result = "resolved"`	Observed:G9TraceClawevent

G9: Effective Agent Resolution Flow Diagram

Purpose: This step resolves the effective Gateway-side Agent after the Agent / Session consistency checks in G8 have passed. The Gateway calls `resolveSessionAgentId()` using the canonical Session key, the current runtime configuration, and the optional Agent identifier returned by G8. The underlying resolver applies the precedence explicit Agent → Session-key Agent → fallback Agent → default Agent. For R1, no explicit Agent is supplied, while the canonical Session key encodes `main`. Therefore, the Session-key Agent branch is selected and the effective Agent is resolved as `main`.



G10: Session Send-Policy Evaluation

Purpose: This step evaluates whether the resolved Session is permitted to continue through the `chat.send` path. The Gateway calls `resolveSendPolicy()` with the current runtime configuration, resolved SessionEntry, canonical Session key, and Session channel / chat-type metadata. The policy resolver first honors a SessionEntry override, then evaluates the configured Session send policy, including ambiguous-key protection and rule matching. A matching deny decision terminates the request immediately. For R1, the observed policy decision is `allow`, so the deny

gate is not taken and execution continues toward G11. The runtime trace does not identify which internal allow-producing branch generated the final decision.

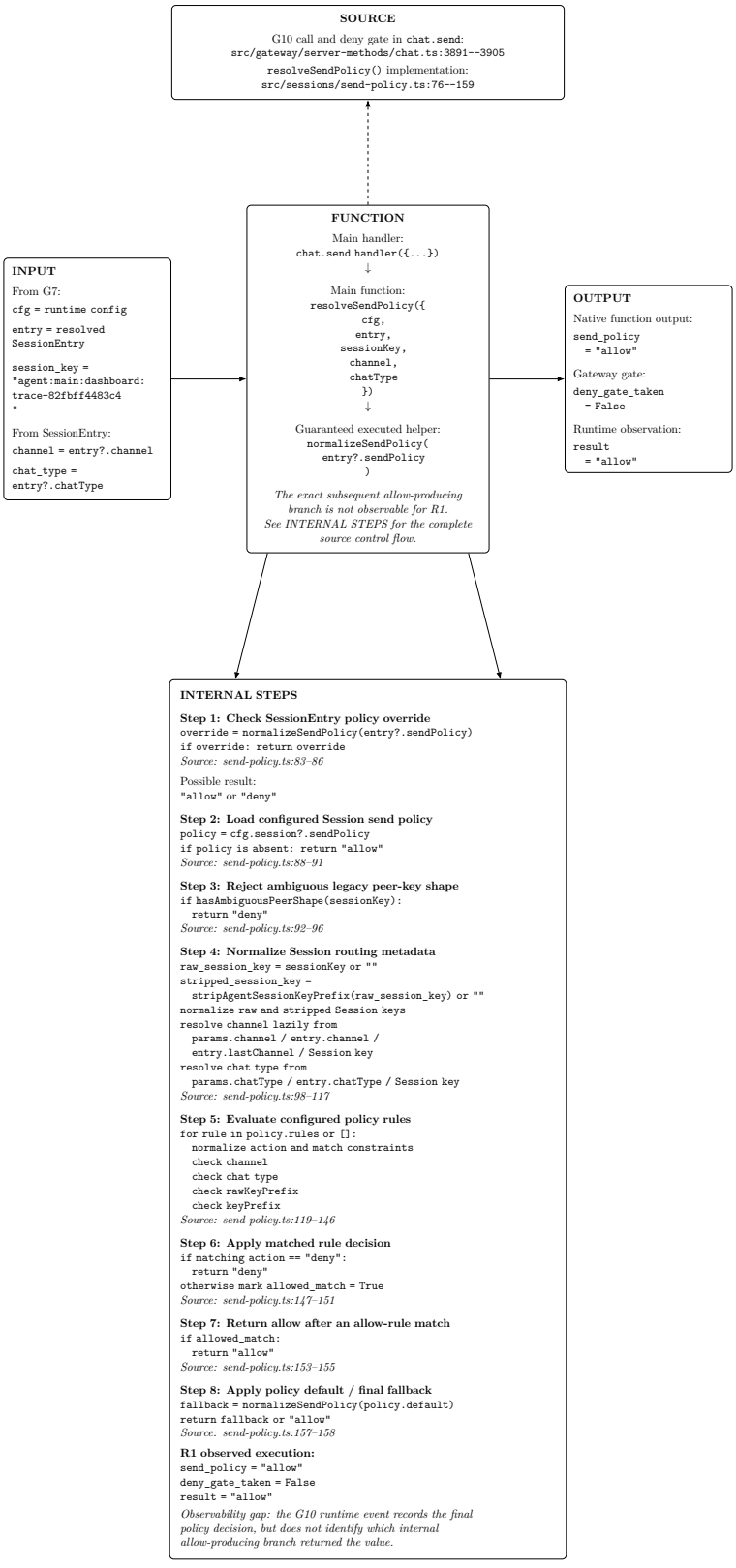
Python-style pseudocode	Exact source / evidence
`send_policy = resolveSendPolicy(`	src/gateway/server-methods/chat.ts:3891--3897
`  cfg=cfg,`	src/gateway/server-methods/chat.ts:3891--3896
`  entry=entry,`	src/gateway/server-methods/chat.ts:3891--3896
`  sessionKey=session_key,`	src/gateway/server-methods/chat.ts:3894
`  channel=entry.channel if entry else None,`	src/gateway/server-methods/chat.ts:3895
`  chatType=entry.chatType if entry else None,`	src/gateway/server-methods/chat.ts:3896
`)`	src/gateway/server-methods/chat.ts:3891--3897
`if send_policy == "deny":`	src/gateway/server-methods/chat.ts:3898--3905
`  return invalid_request("send blocked by session policy ")`	src/gateway/server-methods/chat.ts:3899--3904
`def resolveSendPolicy(params):`	src/sessions/send-policy.ts:76--159
`  override = normalizeSendPolicy(params.entry.sendPolicy )`	src/sessions/send-policy.ts:83
`  if override:`	src/sessions/send-policy.ts:84--86
`  return override`	src/sessions/send-policy.ts:85
`  policy = params.cfg.session.sendPolicy`	src/sessions/send-policy.ts:88
`  if policy is None:`	src/sessions/send-policy.ts:89--91
`  return "allow"`	src/sessions/send-policy.ts:90
`  if hasAmbiguousPeerShape(params.sessionKey):`	src/sessions/send-policy.ts:92--96
`  return "deny"`	src/sessions/send-policy.ts:95
`  raw_session_key = params.sessionKey or ""`	src/sessions/send-policy.ts:98
`  stripped_session_key = stripAgentSessionKeyPrefix( raw_session_key) or ""`	src/sessions/send-policy.ts:99
`  raw_key_norm = normalizeLowercaseStringOrEmpty( raw_session_key)`	src/sessions/send-policy.ts:100
`  stripped_key_norm = normalizeLowercaseStringOrEmpty( stripped_session_key)`	src/sessions/send-policy.ts:101
`  channel = params.channel or entry.channel or entry. lastChannel or derived_channel`	src/sessions/send-policy.ts:102--111
`  chat_type = params.chatType or entry.chatType or derived_chat_type`	src/sessions/send-policy.ts:112--117

```
   allowed_match = False                                          src/sessions/send-policy.ts:119
   for rule in policy.rules or []:                                src/sessions/send-policy.ts:120--151
      action = normalizeSendPolicy(rule.action) or "allow
↪ "                                                               src/sessions/send-policy.ts:124
      match_channel = normalize(rule.match.channel)              src/sessions/send-policy.ts:125--126
      match_chat_type = normalize(rule.match.chatType)           src/sessions/send-policy.ts:125--127
      match_prefix = normalize(rule.match.keyPrefix)             src/sessions/send-policy.ts:125--128
      match_raw_prefix = normalize(rule.match.
↪ rawKeyPrefix)                                                   src/sessions/send-policy.ts:125--129
      if channel constraint does not match: continue             src/sessions/send-policy.ts:131--133
      if chat-type constraint does not match: continue           src/sessions/send-policy.ts:134--136
      if raw-key prefix does not match: continue                 src/sessions/send-policy.ts:137--139
      if normalized keyPrefix does not match: continue           src/sessions/send-policy.ts:140--146
      if action == "deny": return "deny"                         src/sessions/send-policy.ts:147--149
      allowed_match = True                                       src/sessions/send-policy.ts:150
   if allowed_match:                                              src/sessions/send-policy.ts:153--155
      return "allow"                                              src/sessions/send-policy.ts:154
   fallback = normalizeSendPolicy(policy.default)                 src/sessions/send-policy.ts:157
   return fallback or "allow"                                     src/sessions/send-policy.ts:158
R1: session_key =                                                 Observed:
                                                                  canonicalSessionkeycarriedfromG7
   "agent:main:dashboard:trace-82fbff4483c4"                      Observed:G7canonicalSessionkey
R1: entry = NOT RECORDED                                          NotObserved:continuedcontrolflowisSource
                                                                  Derivedevidenceonly
R1: send_policy = "allow"                                         Observed:G10TraceClawevent
R1: deny_gate_taken = False                                       SourceDerived:
                                                                  sendPolicy!="deny"atchat.ts:3898
R1: result = "allow"                                              Observed:G10TraceClawevent
R1: exact internal allow branch = unknown                         NotObserved:Observabilitygap;G10eventrec
                                                                  ordsfinaldecisiononly
```

G10: Session Send-Policy Evaluation Flow Diagram

Purpose: This step determines whether the resolved Session is permitted to continue through the `chat.send` path. The Gateway evaluates the Session send policy using the current configuration, resolved SessionEntry, canonical Session key, and Session channel / chat-type metadata.

A `deny` result immediately terminates the request. For R1, the runtime-observed policy result is `allow`, so the deny gate is not taken and execution continues to G11. The available trace does not identify which internal allow-producing branch generated the final decision.



G11: Idempotency and Existing-Run Guard

Purpose: This step determines whether the incoming `chat.send` request represents a genuinely new dispatch or a retry / duplicate of an already completed, aborted, pending, active, or queued

run. The Gateway checks multiple run-state registries using the client-provided run ID and short-circuits when an existing state is found. Only a request that passes all existing-run guards is allowed to proceed as a new dispatch. Before entering Session lifecycle admission, the Gateway also checks whether Session routing has changed and whether the resolved Session is permitted to start new work. For R1, TraceClaw records `dedupeDecision = "new_dispatch"` and `result = "pass"`, so no existing-run short-circuit handles the request and execution continues toward G12.

Python-style pseudocode	Exact source / evidence
`cached = context.dedupe.get(`	src/gateway/server-methods/chat.ts: 3936--3942
`  "chat:" + client_run_id`	src/gateway/server-methods/chat.ts:3936
`)`	src/gateway/server-methods/chat.ts:3936
`if cached is not None:`	src/gateway/server-methods/chat.ts: 3937--3942
`  respond(cached result)`	src/gateway/server-methods/chat.ts: 3938--3941
`  return`	src/gateway/server-methods/chat.ts: 3938--3942
`abort_marker = context.chatAbortedRuns.get(client_run_id)`	src/gateway/server-methods/chat.ts: 3944--3965
`if abort_marker is not None:`	src/gateway/server-methods/chat.ts: 3945--3965
`  aborted_at = chatAbortMarkerTimestampMs(abort_marker)`	src/gateway/server-methods/chat.ts: 3945--3965
`  payload = buildAbortedChatSendPayload(...)`	src/gateway/server-methods/chat.ts: 3945--3965
`  setGatewayDedupeEntry(...)`	src/gateway/server-methods/chat.ts: 3945--3965
`  respond(cached aborted result)`	src/gateway/server-methods/chat.ts: 3945--3965
`  return`	src/gateway/server-methods/chat.ts: 3945--3965
`pending_chat_send = readPreRegisteredRun(`	src/gateway/server-methods/chat.ts: 3967--3978
`  key=pending_chat_send_key,`	src/gateway/server-methods/chat.ts: 3967--3978
`  entry=context.dedupe.get(pending_chat_send_key),`	src/gateway/server-methods/chat.ts: 3967--3978
`  keyPrefix=PENDING_CHAT_SEND_DEDUPE_PREFIX`	src/gateway/server-methods/chat.ts: 3967--3978

```
)
                                                    src/gateway/server-methods/chat.ts:
                                                    3967--3978
if pending_chat_send is not None:
                                                    src/gateway/server-methods/chat.ts:
                                                    3967--3978
   respond(status="in_flight", cached=True)
                                                    src/gateway/server-methods/chat.ts:
                                                    3967--3978
   return
                                                    src/gateway/server-methods/chat.ts:
                                                    3967--3978
```

```
active_existing =
                                                    src/gateway/server-methods/chat.ts:
                                                    3980--3987
   context.chatAbortControllers.get(client_run_id)
                                                    src/gateway/server-methods/chat.ts:
                                                    3980--3987
if active_existing is not None:
                                                    src/gateway/server-methods/chat.ts:
                                                    3980--3987
   respond(status="in_flight", cached=True)
                                                    src/gateway/server-methods/chat.ts:
                                                    3980--3987
   return
                                                    src/gateway/server-methods/chat.ts:
                                                    3980--3987
```

```
if context.chatQueuedTurns?.has(client_run_id):
                                                    src/gateway/server-methods/chat.ts:
                                                    3988--3994
   respond(status="in_flight", cached=True)
                                                    src/gateway/server-methods/chat.ts:
                                                    3988--3994
   return
                                                    src/gateway/server-methods/chat.ts:
                                                    3988--3994
```

```
if sessionRoutingChanged(cfg):
                                                    src/gateway/server-methods/chat.ts:
                                                    3995--4000
   respondSessionRoutingChanged()
                                                    src/gateway/server-methods/chat.ts:
                                                    3995--4000
   return
                                                    src/gateway/server-methods/chat.ts:
                                                    3995--4000
```

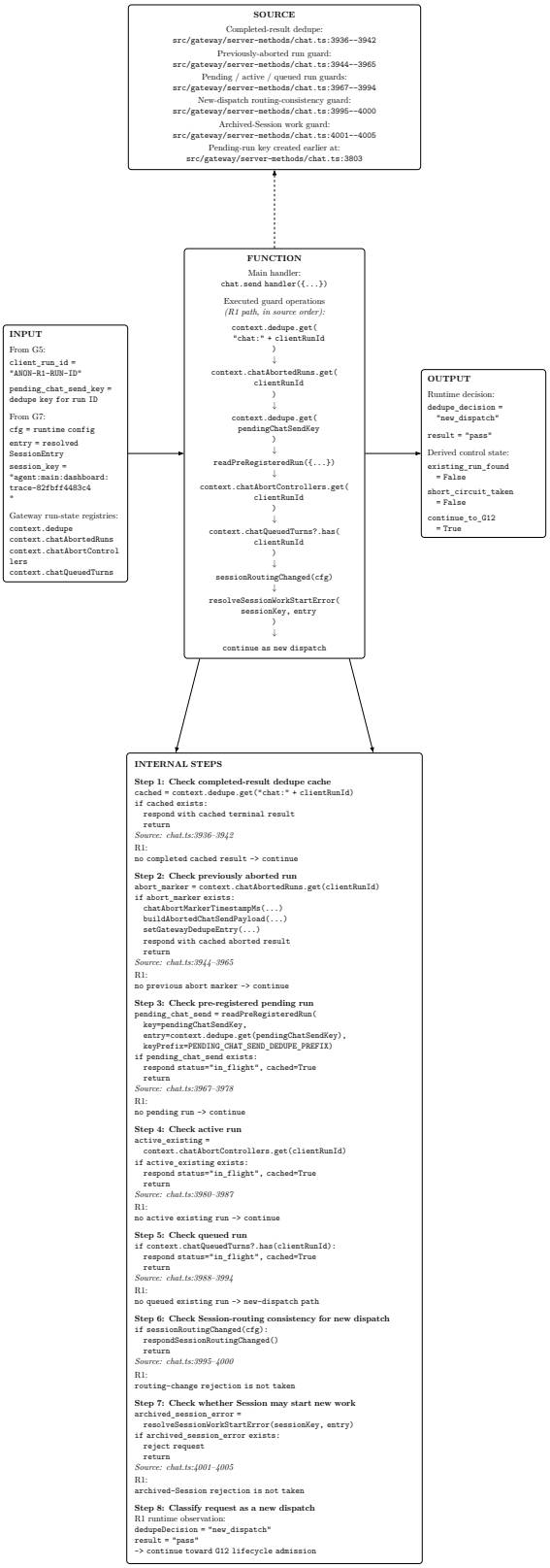
```
archived_session_error =
                                                    src/gateway/server-methods/chat.ts:
                                                    4001--4005
   resolveSessionWorkStartError(session_key, entry)
                                                    src/gateway/server-methods/chat.ts:
                                                    4001--4005
if archived_session_error is not None:
                                                    src/gateway/server-methods/chat.ts:
                                                    4001--4005
   reject request
                                                    src/gateway/server-methods/chat.ts:
                                                    4001--4005
```

```
R1: client_run_id =
    "ANON-R1-RUN-ID"
R1: completed cached result = None
R1: previous abort marker = None
R1: pending run = None
R1: active existing run = None
R1: queued existing run = False
R1: dedupe_decision = "new_dispatch"
R1: result = "pass"
R1: continue toward lifecycle admission
```

```
Observed:carriedfromG5
Observed:R1runID
SourceDerived:
completed-resultshortcircuitnottaken
SourceDerived:
previously-abortedbranchnottaken
SourceDerived:
pendingin-flightbranchnottaken
SourceDerived:
activein-flightbranchnottaken
SourceDerived:
queuedin-flightbranchnottaken
Observed:G11TraceClawevent
Observed:G11TraceClawevent
SourceDerived:noG11shortcircuit
```

G11: Idempotency and Existing-Run Guard Flow Diagram

Purpose: This step determines whether the incoming `chat.send` request is a genuinely new dispatch or a retry / duplicate of an existing run. The Gateway checks completed, aborted, pending, active, and queued run state using the request run ID. Any existing state short-circuits the request. A request that survives these guards is treated as a new dispatch and is additionally checked for Session-routing consistency and Session work eligibility before lifecycle admission. For R1, TraceClaw records `dedupeDecision = "new_dispatch"` and `result = "pass"`, so execution continues toward G12.



G12: Session Work Admission and Revalidation

Purpose: This step converts the new-dispatch candidate from G11 into an admitted Session-scoped run. The Gateway first captures the current Agent lifecycle generation and creates a temporary pending reservation. It then invokes `beginSessionWorkAdmission(...)` using the resolved Session key and any available backing identity. While acquiring the admission lease, the Gateway revalidates reservation ownership, lifecycle generation, reservation expiry, latest Session state, routing consistency, backing Session identity when present, and archival status.

If these checks succeed, an active chat AbortController is registered and the run is admitted. Post-admission race checks are then performed before the Agent run context is claimed. For R1, TraceClaw records `admissionDecision = "admitted"`, `result = "admitted"`, and `latestSessionRevalidated = true`.

Python-style pseudocode	Exact source / evidence
`lifecycle_generation = getAgentEventLifecycleGeneration()`	src/gateway/server-methods/chat.ts:4025
# helper returns the current Agent-event lifecycle generation	src/infra/agent-events.ts:195--197
`pending_attempt_id = randomUUID()`	src/gateway/server-methods/chat.ts:4026
`pending_expires_at_ms = resolveChatRunExpiresAtMs(`	src/gateway/server-methods/chat.ts:4027
`  now=now, timeoutMs=timeout_ms`	src/gateway/server-methods/chat.ts:4027
`)`	src/gateway/server-methods/chat.ts:4027
`context.dedupe.set(pending_chat_send_key, {...reservation ...})`	src/gateway/server-methods/chat.ts: 4030--4047
# reservation contains runId, attemptId, sessionKey,	src/gateway/server-methods/chat.ts: 4030--4047
# owner connection/device, expiry, and turn kind	src/gateway/server-methods/chat.ts: 4030--4047
`admitted_session_id = backing_session_id or client_run_id`	src/gateway/server-methods/chat.ts:4061
`admitted_run_abort = None`	src/gateway/server-methods/chat.ts: 4062--4063
`reservation_superseded = False`	src/gateway/server-methods/chat.ts:4064
`gateway_work_admission = await beginSessionWorkAdmission(`	src/gateway/server-methods/chat.ts: 4067--4163
`  scope=store_path,`	src/gateway/server-methods/chat.ts:4068
`  identities=[session_key, backing_session_id],`	src/gateway/server-methods/chat.ts:4069
`  assertAllowed=assert_allowed,`	src/gateway/server-methods/chat.ts: 4070--4155
`  onInterrupt=on_interrupt`	src/gateway/server-methods/chat.ts: 4157--4162
`)`	src/sessions/session-lifecycle-admission .ts:327--378
`def assert_allowed():`	src/gateway/server-methods/chat.ts: 4070--4155
`  if context.chatAbortedRuns.has(client_run_id):`	src/gateway/server-methods/chat.ts: 4071--4073
`    return`	src/gateway/server-methods/chat.ts: 4071--4073

```
   pending = readPreRegisteredRun(...)                                  src/gateway/server-methods/chat.ts:
                                                                        4074--4078
   if pending exists and pending.attemptId !=                           src/gateway/server-methods/chat.ts:
↪ pending_attempt_id:                                                   4079--4085
      reservation_superseded = True                                     src/gateway/server-methods/chat.ts:
                                                                        4079--4085
      return                                                            src/gateway/server-methods/chat.ts:
                                                                        4079--4085
   if pending is None:                                                  src/gateway/server-methods/chat.ts:
                                                                        4086--4093
      check terminal result or existing AbortController                 src/gateway/server-methods/chat.ts:
                                                                        4086--4093
      mark reservation superseded if another run won the                src/gateway/server-methods/chat.ts:
↪ race                                                                  4086--4093
```

```
   if lifecycle_generation !=                                           src/gateway/server-methods/chat.ts:
↪ getAgentEventLifecycleGeneration():                                   4094--4103
      writePreRegisteredChatAbort(stopReason="restart")                 src/gateway/server-methods/chat.ts:
                                                                        4094--4103
      return                                                            src/gateway/server-methods/chat.ts:
                                                                        4094--4103
   if pending is None or reservation is expired:                        src/gateway/server-methods/chat.ts:
                                                                        4104--4116
      writePreRegisteredChatAbort(stopReason="timeout")                 src/gateway/server-methods/chat.ts:
                                                                        4104--4116
      return                                                            src/gateway/server-methods/chat.ts:
                                                                        4104--4116
```

```
   latest_session = loadSessionEntry(raw_session_key,                   src/gateway/server-methods/chat.ts:4117
↪ session_load_options)
   if sessionRoutingChanged(latest_session.cfg):                        src/gateway/server-methods/chat.ts:
                                                                        4118--4120
      raise SESSION_ROUTING_CHANGED                                     src/gateway/server-methods/chat.ts:
                                                                        4118--4120
   latest_entry = latest_session.entry                                  src/gateway/server-methods/chat.ts:4121
   if entry existed and latest_entry disappeared:                       src/gateway/server-methods/chat.ts:
                                                                        4122--4124
      reject Session deleted while starting work                        src/gateway/server-methods/chat.ts:
                                                                        4122--4124
```

```
   if backing_session_id and latest_entry.sessionId                     src/gateway/server-methods/chat.ts:
                                                                        4125--4132
```

```
      and latest_entry.sessionId != backing_session_id:
                                                        src/gateway/server-methods/chat.ts:
                                                        4125--4132
         reject Session changed while starting work
                                                        src/gateway/server-methods/chat.ts:
                                                        4125--4132
   archived_error = resolveSessionWorkStartError(
↪ session_key, latest_entry)                            src/gateway/server-methods/chat.ts:
                                                        4133--4136
   if archived_error: raise error
                                                        src/gateway/server-methods/chat.ts:
                                                        4133--4136
   admitted_session_id =
                                                        src/gateway/server-methods/chat.ts:
                                                        4134--4138
      latest_entry.sessionId or backing_session_id or
↪ client_run_id                                         src/gateway/server-methods/chat.ts:
                                                        4134--4138
```

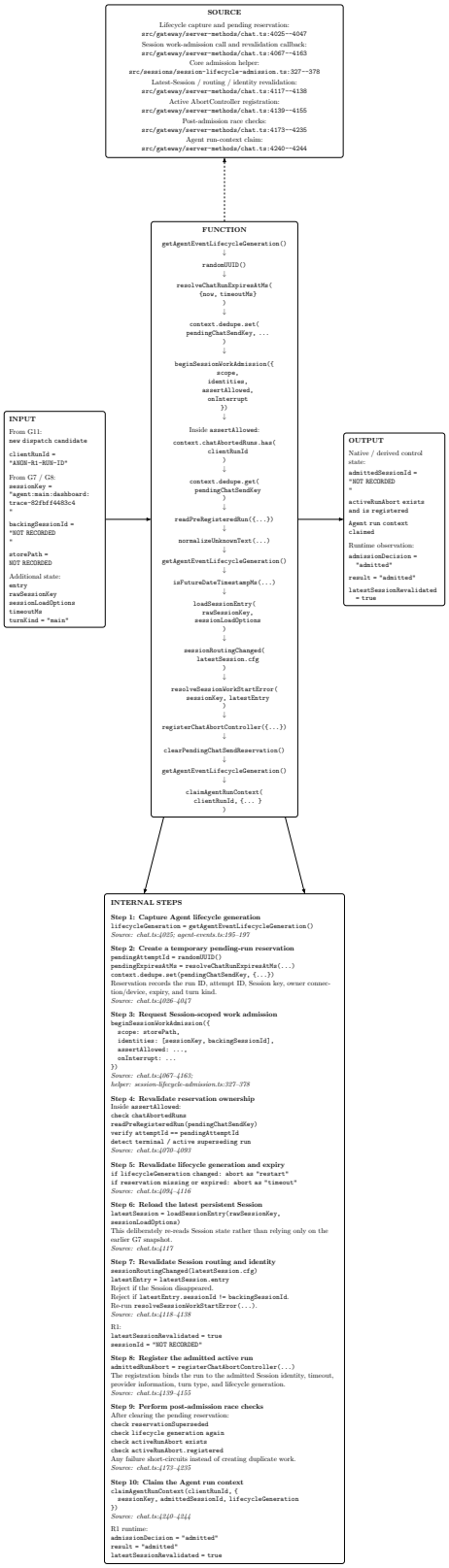
```
   admitted_run_abort = registerChatAbortController(
                                                        src/gateway/server-methods/chat.ts:
                                                        4139--4155
      runId=client_run_id,
                                                        src/gateway/server-methods/chat.ts:
                                                        4139--4155
      sessionId=admitted_session_id,
                                                        src/gateway/server-methods/chat.ts:
                                                        4139--4155
      sessionKey=session_key,
                                                        src/gateway/server-methods/chat.ts:
                                                        4139--4155
      timeoutMs=timeout_ms,
                                                        src/gateway/server-methods/chat.ts:
                                                        4139--4155
      kind="chat-send", turnKind=turn_kind,
                                                        src/gateway/server-methods/chat.ts:
                                                        4139--4155
      lifecycleGeneration=lifecycle_generation
                                                        src/gateway/server-methods/chat.ts:
                                                        4139--4155
   )
                                                        src/gateway/chat-abort.ts:146--...
```

```
clearPendingChatSendReservation()
                                                        src/gateway/server-methods/chat.ts:
                                                        4165--4172
active_run_abort = admitted_run_abort
                                                        src/gateway/server-methods/chat.ts:4173
if reservation_superseded: short-circuit
                                                        src/gateway/server-methods/chat.ts:
                                                        4174--4188
if lifecycle generation changed: abort and short-circuit
                                                        src/gateway/server-methods/chat.ts:
                                                        4189--4212
if active_run_abort is None: reject admission
                                                        src/gateway/server-methods/chat.ts:
                                                        4213--4225
if not active_run_abort.registered: return existing in-
↪ flight run                                            src/gateway/server-methods/chat.ts:
                                                        4226--4235
```

`claimAgentRunContext(client_run_id, {`	src/gateway/server-methods/chat.ts: 4240--4244
`sessionKey=session_key,`	src/gateway/server-methods/chat.ts: 4240--4244
`sessionId=admitted_session_id,`	src/gateway/server-methods/chat.ts: 4240--4244
`lifecycleGeneration=lifecycle_generation`	src/gateway/server-methods/chat.ts: 4240--4244
`})`	src/infra/agent-events.ts:277--...
`R1: client_run_id =`	Observed
`"ANON-R1-RUN-ID"`	Observed
`R1: session_key =`	Observed
`"agent:main:dashboard:trace-82fbff4483c4"`	Observed
`R1: backing_session_id =`	NotObserved:absentfromGatewaytrace
`"NOT RECORDED"`	NotObserved: donotsubstituteAR-layersessionId
`R1: admitted_session_id = NOT RECORDED`	NotObserved: sourcecanfallbacktoclient_run_id
`R1: reservation_superseded = False`	SourceDerived:G12successfuladmissionpath
`R1: active_run_abort exists and is registered`	SourceDerived:post-admissionguardspassed
`R1: admissionDecision = "admitted"`	Observed:G12TraceClawevent
`R1: result = "admitted"`	Observed:G12TraceClawevent
`R1: latestSessionRevalidated = true`	Observed:G12TraceClawevent

G12: Session Work Admission and Revalidation Flow Diagram

Purpose: This step converts the new-dispatch candidate into an admitted Session-scoped run. The Gateway creates a temporary run reservation, acquires a Session work-admission lease, revalidates the latest Session and lifecycle state, registers the active run AbortController, performs post-admission race checks, and finally claims the Agent run context. For R1, TraceClaw records the revalidation outcome as `admissionDecision = "admitted"`, `result = "admitted"`, and `latestSessionRevalidated = true`; the trace does not separately archive a stronger object-identity check for the latest Session.



G13: Runtime Message Context Construction

Purpose: This step converts the message, Session identity, Agent identity, routing metadata, command state, and authenticated Gateway-client state prepared by the preceding stages into a concrete `MsgContext`. The Gateway first derives the command-facing and Agent-facing forms of the message, extracts the originating delivery route, and then constructs `ctx` inline. The resulting context contains the canonical Session key, effective Agent ID, internal message channel, command-turn state, run identity, and Gateway client scopes. Optional staged-media fields are

added only when media offloads exist, and managed-media fields are merged immediately before dispatch. G13 does not perform a new Session lookup, Agent selection, send-policy decision, or model invocation. For R1, TraceClaw records `event = "runtime_context_constructed"`, `AgentId = "main"`, scope `operator.write`, and `result = "constructed"`.

Python-style pseudocode	Exact source / evidence
Prepare message and command state	**Source**
`trimmed_message = parsed_message.trim()`	src/gateway/server-methods/chat.ts:4475
`command_body = parsed_message`	chat.ts:4476
`command_source =`	chat.ts:4477--4478
`  "text" if not suppress_command_interpretation`	chat.ts:4477--4478
`  and trimmed_message.startswith("/") else None`	chat.ts:4477--4478
`message_for_agent =`	chat.ts:4479--4481
`  system_provenance_receipt + parsed_message`	chat.ts:4479--4481
`  if system_provenance_receipt else parsed_message`	chat.ts:4479--4481
Resolve message ownership and route metadata	**Source**
`queued_followup_owner_device_id = normalizeOptionalText ↪ (...)`	chat.ts:4482
`queued_followup_owner_conn_id = normalizeOptionalText ↪ (...)`	chat.ts:4483
`queued_followup_owner_key = device or connection owner`	chat.ts:4484--4488
`originating_channel, originating_to, account_id,`	chat.ts:4489--4495
`  message_thread_id, explicit_deliver_route = ↪ originating_route`	chat.ts:4489--4495
Construct the core MsgContext	**Source**
`ctx = MsgContext(`	src/gateway/server-methods/chat.ts:4503--4545
`  Body=message_for_agent,`	chat.ts:4504
`  BodyForAgent=message_for_agent,`	chat.ts:4505
`  BodyForCommands=command_body,`	chat.ts:4506
`  RawBody=parsed_message,`	chat.ts:4507
`  CommandBody=command_body,`	chat.ts:4508
`  InputProvenance=system_input_provenance,`	chat.ts:4509
`  SessionKey=session_key,`	chat.ts:4510
`  AgentId=agent_id,`	chat.ts:4511
`  Provider=INTERNAL_MESSAGE_CHANNEL,`	chat.ts:4512
`  Surface=INTERNAL_MESSAGE_CHANNEL,`	chat.ts:4513

```
   OriginatingChannel=originating_channel,                    chat.ts:4514
   OriginatingTo=originating_to,                              chat.ts:4515
   ExplicitDeliverRoute=explicit_deliver_route,               chat.ts:4516
   AccountId=account_id,                                      chat.ts:4517
   MessageThreadId=message_thread_id,                         chat.ts:4518
   ChatType="direct",                                         chat.ts:4519
```

Construct command-turn state	Source

```
   CommandSource=command_source if command_source exists,     chat.ts:4520
   CommandAuthorized=not suppress_command_interpretation,     chat.ts:4521
   CommandTurn=(                                              chat.ts:4522--4534
      TextSlashTurn(source=command_source, authorized=        chat.ts:4522--4528
↪ True)
      if command_source else                                  chat.ts:4522--4534
      NormalTurn(source="message", authorized=False)          chat.ts:4529--4534
   ),                                                         chat.ts:4522--4534
```

Attach run and authenticated client state	Source

```
   MessageSid=client_run_id,                                  chat.ts:4535
   ApprovalReviewerDeviceId=                                  chat.ts:4536
↪ queued_followup_owner_device_id,
   if not isOperatorUiClient(client_info):                    chat.ts:4537--4543
      attach SenderId / SenderName / SenderUsername           chat.ts:4537--4543
   GatewayClientScopes=client.connect.scopes or [],           chat.ts:4544
)                                                             chat.ts:4503--4545
```

Optional staged-media enrichment	Source

```
if len(media_path_offload_paths) > 0:                         src/gateway/server-methods/
                                                              chat.ts:4548--4559
   ctx.MediaPath = media_path_offload_paths[0]                chat.ts:4554
   ctx.MediaPaths = media_path_offload_paths                  chat.ts:4555
   ctx.MediaType = media_path_offload_types[0]                chat.ts:4556
   ctx.MediaTypes = media_path_offload_types                  chat.ts:4557
   ctx.MediaWorkspaceDir =                                    chat.ts:4558
↪ media_path_offload_workspace_dir
   ctx.MediaStaged = True                                     chat.ts:4559
```

Managed-media context merge	Source

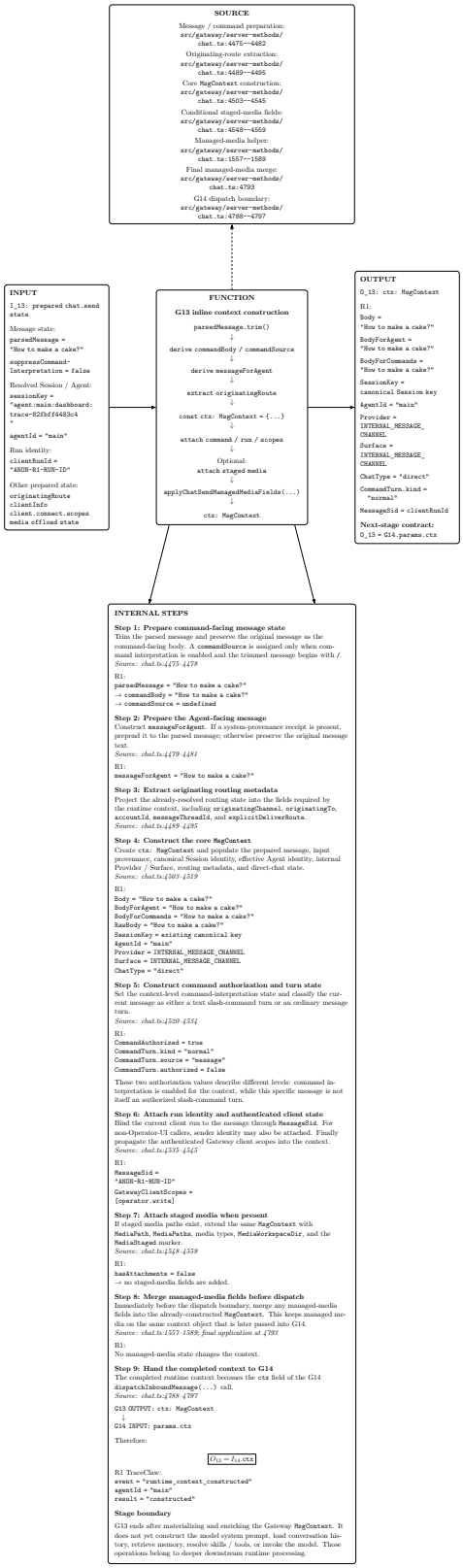
`def resolveChatSendManagedMediaFields(...):`	src/gateway/server-methods/chat.ts:1482--1497
`def applyChatSendManagedMediaFields(ctx, fields):`	src/gateway/server-methods/chat.ts:1498--1515
`applyChatSendManagedMediaFields(`	src/gateway/server-methods/chat.ts:4793
`ctx, await plugin_bound_media_fields_promise`	chat.ts:4793
`)`	chat.ts:4793

G14 boundary	Source
`dispatch_result = await dispatchInboundMessage(...)`	src/gateway/server-methods/chat.ts:4788--4797

R1 concrete state	Evidence
`R1: parsed_message = "How to make a cake?"`	Observed;SourceDerivedforsourcerationale
`R1: message_for_agent = "How to make a cake?"`	SourceDerived
`R1: command_source = None`	SourceDerived:non-slashmessage
`R1: session_key =`	Observed
`"agent:main:dashboard:trace-82fbff4483c4"`	Observed
`R1: agent_id = "main"`	Observed
`R1: client_run_id =`	Observed
`"ANON-R1-RUN-ID"`	Observed
`R1: ChatType = "direct"`	SourceDerived
`R1: CommandTurn.kind = "normal"`	SourceDerived
`R1: CommandTurn.authorized = False`	SourceDerived
`R1: GatewayClientScopes = [operator.write]`	Observed:G13TraceClawevent
`R1: result = "constructed"`	Observed:G13TraceClawevent

G13: Runtime Message Context Construction Flow Diagram

Purpose: This step packages the prepared message, canonical Session identity, effective Agent identity, routing metadata, command state, run identity, authenticated client scopes, and optional media metadata into the `MsgContext` consumed by the downstream G14 dispatch path. For R1, the resulting context retains `"How to make a cake?"`, the existing canonical Session key, and `AgentId = "main"`. TraceClaw records the runtime context as successfully constructed.



G14: Inbound Message Dispatch

Purpose: This step transfers the Gateway-constructed `MsgContext` into OpenClaw's auto-reply dispatch pipeline. The Gateway executes `dispatchInboundMessage(...)` inside the admitted Session-work region and the native `gateway.chat_send.dispatch_inbound` diagnostic span. The dispatch function propagates any runtime tool restriction, initializes and binds downstream reply-run state, invokes G15 to finalize the inbound context, records optional message-received diagnostics, installs reply-payload delivery hooks, and invokes G16 through

`dispatchReplyFromConfig(...)`. When downstream dispatch returns, G14 reconciles the result with dispatcher cancellation and failure counts. G15 and G16 are nested operations inside G14 rather than independent parallel stages. For R1, `dispatch_inbound_entered` directly records `result=entered`, the run ID, Session key, and `agentId=main`. Provider/model are recorded in the saved runtime metadata. The native diagnostic span is a source-level boundary, not a separately archived R1 span; the complete dispatch return payload remains partly source-derived or unrecorded.

Python-style pseudocode	Exact source / evidence
Gateway G14 call site	**Source**
`gateway_work_admission.run(`	src/gateway/server-methods/ chat.ts:4788--4797
`  measureDiagnosticsTimelineSpan(`	chat.ts:4788--4797
`  "gateway.chat_send.dispatch_inbound",`	chat.ts:4788--4797
`  async dispatch_callback`	chat.ts:4788--4797
`  )`	chat.ts:4788--4797
`)`	chat.ts:4788--4797
`dispatch_result = await dispatchInboundMessage(`	src/gateway/server-methods/ chat.ts:4788--4797
`  ctx=ctx,`	chat.ts:4788--4797
`  cfg=cfg,`	chat.ts:4788--4797
`  dispatcher=dispatcher,`	chat.ts:4788--4797
`  ...`	chat.ts:4788--4797
`)`	chat.ts:4788--4797
Main G14 function	**Source**
`async def dispatchInboundMessage(params):`	src/auto-reply/ dispatch.ts:528--537
`  reply_options = applyRuntimeToolsAllow(`	dispatch.ts:538
`  params.replyOptions, params.toolsAllow`	dispatch.ts:538
`  )`	dispatch.ts:538
Propagate runtime tool restriction	**Source**
`def applyRuntimeToolsAllow(reply_options, tools_allow):`	src/auto-reply/ dispatch.ts:69--80
`  if tools_allow is None:`	dispatch.ts:69--80
`  return reply_options`	dispatch.ts:69--80
`  return reply_options with toolsAllow=tools_allow`	dispatch.ts:69--80
Initialize and bind reply-run state	**Source**

```
   reply_payload_run_state =                                  src/auto-reply/
                                                              dispatch.ts:539--541
      params.replyPayloadRunState or                          dispatch.ts:539
      {"runId": reply_options.runId}                          dispatch.ts:540
   reply_options_with_run_state =                             dispatch.ts:542
      bindReplyPayloadRunState(reply_options,                 dispatch.ts:542
↪ reply_payload_run_state)
def bindReplyPayloadRunState(reply_options, run_state):       src/auto-reply/
                                                              dispatch.ts:410--422
   wrap onAgentRunStart(runId)                                dispatch.ts:410--422
   run_state.runId = runId                                    dispatch.ts:410--422
```

Invoke G15	Source

```
   finalized = measureDiagnosticsTimelineSpanSync(            src/auto-reply/
                                                              dispatch.ts:544--551
      "auto_reply.finalize_context",                          dispatch.ts:544--551
      lambda: finalizeInboundContext(params.ctx),             dispatch.ts:544--551
      phase="agent-turn", config=params.cfg,                  dispatch.ts:544--551
      attributes=buildDispatchTimelineAttributes(params.      dispatch.ts:544--551
↪ ctx)
   )                                                          dispatch.ts:544--551
```

Optional inbound diagnostics	Source

```
   if isDiagnosticsEnabled(params.cfg):                       src/auto-reply/
                                                              dispatch.ts:552--561
      logMessageReceived(                                     dispatch.ts:552--561
         sessionKey=finalized.SessionKey,                     dispatch.ts:552--561
         channel=finalized.Surface or finalized.Provider,     dispatch.ts:552--561
         messageId=finalized.MessageSid or fallback           dispatch.ts:552--561
      )                                                       dispatch.ts:552--561
```

Install reply-payload delivery hook	Source

```
   installReplyPayloadSendingBeforeDeliver(                   src/auto-reply/
                                                              dispatch.ts:561
      params.dispatcher, finalized,                           dispatch.ts:561
↪ reply_payload_run_state
   )                                                          dispatch.ts:561
def installReplyPayloadSendingBeforeDeliver(...):             src/auto-reply/
                                                              dispatch.ts:424--438
```

Invoke G16	Source
result = await withReplyDispatcher(	src/auto-reply/ dispatch.ts:562--582
dispatcher=params.dispatcher,	dispatch.ts:562--582
run=lambda: measureDiagnosticsTimelineSpan(	dispatch.ts:562--582
"auto_reply.dispatch_reply_from_config",	dispatch.ts:562--582
lambda: dispatchReplyFromConfig(	dispatch.ts:562--582
ctx=finalized,	dispatch.ts:562--582
cfg=params.cfg,	dispatch.ts:562--582
dispatcher=params.dispatcher,	dispatch.ts:562--582
replyOptions=reply_options_with_run_state,	dispatch.ts:562--582
replyResolver=params.replyResolver	dispatch.ts:562--582
)	dispatch.ts:562--582
)	dispatch.ts:562--582
)	dispatch.ts:562--582

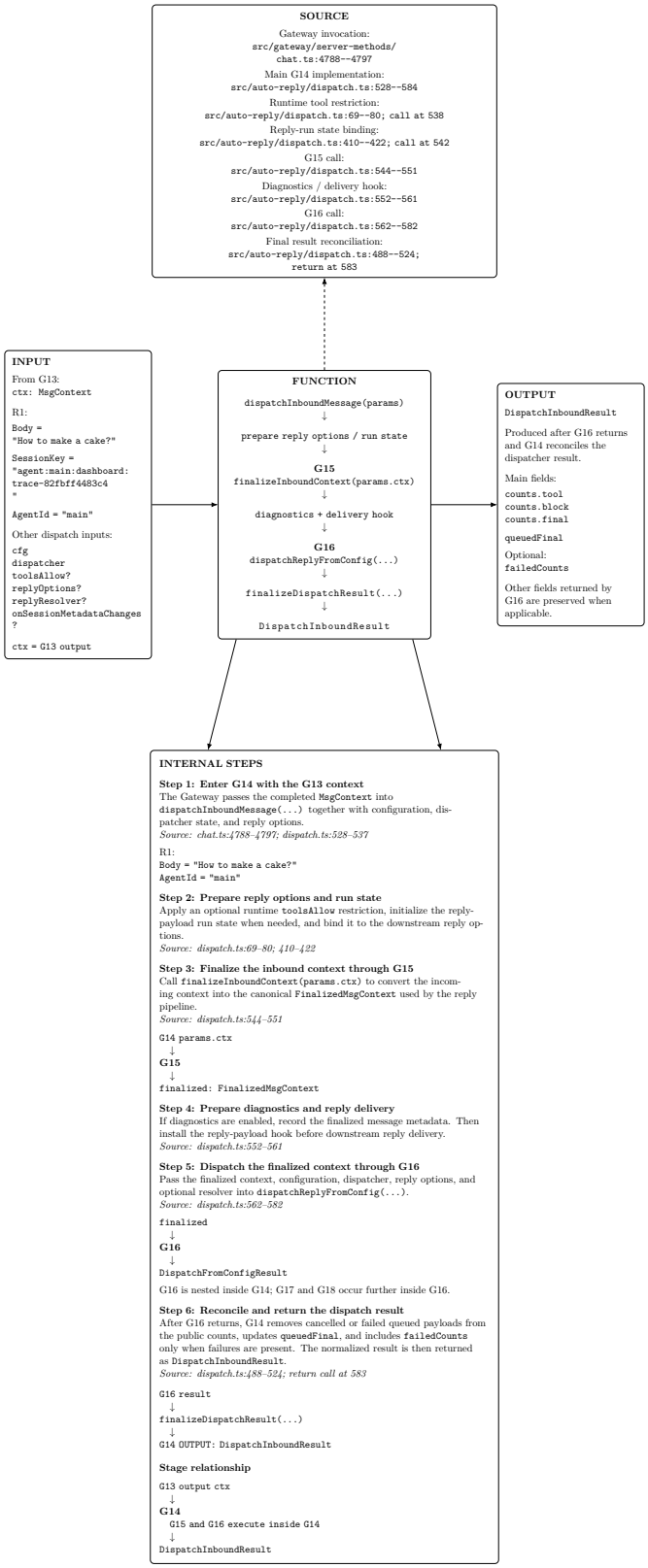
Reconcile the returned dispatch result	Source
def finalizeDispatchResult(result, dispatcher):	src/auto-reply/ dispatch.ts:488--524
cancelled_counts = dispatcher.getCancelledCounts()	dispatch.ts:488--524
failed_counts = dispatcher.getFailedCounts()	dispatch.ts:488--524
if no cancelled or failed counts: return result	dispatch.ts:488--524
reconcile tool / block / final counts	dispatch.ts:488--524
return reconciled result	dispatch.ts:488--524
return finalizeDispatchResult(result, params.dispatcher)	src/auto-reply/ dispatch.ts:583

R1 concrete state	Evidence
R1: run_id =	Observed
"ANON-R1-RUN-ID"	Observed
R1: session_key =	Observed
"agent:main:dashboard:trace-82fbff4483c4"	Observed
R1: agent_id = "main"	Observed
R1: provider = "openai"	Observed

`R1: model = "gpt-5.5"`	`Observed`
`R1: has_attachments = False`	`Observed`
`source diagnostic span = "gateway.chat_send. ↪ dispatch_inbound"`	`SourceDerived:chat.ts: 4790--4792;nativespannotarchived`

G14: Inbound Message Dispatch Flow Diagram

Purpose: This step receives the `MsgContext` prepared by G13 and sends it into the auto-reply pipeline. G14 prepares the reply runtime state, finalizes the inbound context through G15, invokes G16 for reply dispatch, and reconciles the returned result before producing `DispatchInboundResult`.



G15: Inbound Context Finalization

Purpose: This step canonicalizes the `MsgContext` created by G13 before it enters configuration-aware reply dispatch. G15 is called synchronously inside G14 through `finalizeInboundContext(...)` and is wrapped by the native diagnostic span `auto_reply.finalize_context`. The function projects supplemental context, normalizes inbound text and system-tag content, canonicalizes `ChatType`, resolves the effective Agent-facing and command-facing message bodies, derives the conversation label, canonicalizes command authorization and command turn state, and

aligns media-type metadata when media is present. G15 does not perform a new Session lookup, Agent selection, model selection, memory retrieval, skill loading, tool execution, or model inference. Its output is a `FinalizedMsgContext` consumed by G16. For R1, the ordinary direct text message and the previously resolved Session / Agent identity remain stable through this deterministic normalization path. The saved R1 trace contains three `inbound_context_finalized` events with `result=finalized` and `runtimeObserved=true`: the stage outcome is OBSERVED. The complete `FinalizedMsgContext` is not serialized field-by-field; detailed context contents combine observed event fields with source mapping.

Python-style pseudocode	Exact source / evidence
G15 invocation from G14	**Source**
`finalized = measureDiagnosticsTimelineSpanSync(`	src/auto-reply/ dispatch.ts:544--551
`  "auto_reply.finalize_context",`	dispatch.ts:544--551
`  lambda: finalizeInboundContext(params.ctx),`	dispatch.ts:544--551
`  phase="agent-turn",`	dispatch.ts:544--551
`  attributes=buildDispatchTimelineAttributes(params.ctx)`	dispatch.ts:544--551
`)`	dispatch.ts:544--551
Main G15 function	**Source**
`def finalizeInboundContext(ctx):`	src/auto-reply/reply/ inbound-context.ts:76--175
Project supplemental context	**Source**
`def applySupplementalContext(ctx):`	src/auto-reply/reply/ inbound-context.ts:47--74
`  project supported supplemental fields into ctx`	inbound-context.ts:47--74
`finalized = applySupplementalContext(ctx)`	src/auto-reply/reply/ inbound-context.ts:81
Normalize inbound text and system-tag content	**Source**
`normalize inbound text / newline representation`	src/auto-reply/reply/ inbound-context.ts:83--97
`sanitize untrusted system-tag content`	inbound-context.ts:83--97
`preserve trusted GroupSystemPrompt handling`	inbound-context.ts:83--97
Normalize ChatType	**Source**
`chat_type = normalizeChatType(finalized.ChatType)`	src/auto-reply/reply/ inbound-context.ts:99--102
`finalized.ChatType = canonical chat_type when available`	inbound-context.ts:99--102
Resolve Agent-facing and command-facing bodies	**Source**
`body_for_agent = resolve normalized Agent-facing body`	src/auto-reply/reply/ inbound-context.ts:104--113

`finalized.BodyForAgent = body_for_agent`	inbound-context.ts:104--113
`body_for_commands = resolve normalized command-facing body`	src/auto-reply/reply/ inbound-context.ts:114--123
`finalized.BodyForCommands = body_for_commands`	inbound-context.ts:114--123

Resolve conversation label	Source
`conversation_label = resolveConversationLabel(...)`	src/auto-reply/reply/ inbound-context.ts:125--133
`finalized.ConversationLabel = conversation_label`	inbound-context.ts:125--133

Canonicalize command state	Source
`normalize command authorization with default-deny semantics`	src/auto-reply/reply/ inbound-context.ts:135--143
`command_turn = resolveCommandTurnContext(finalized)`	inbound-context.ts:135--143
`finalized.CommandTurn = canonical command_turn`	inbound-context.ts:135--143

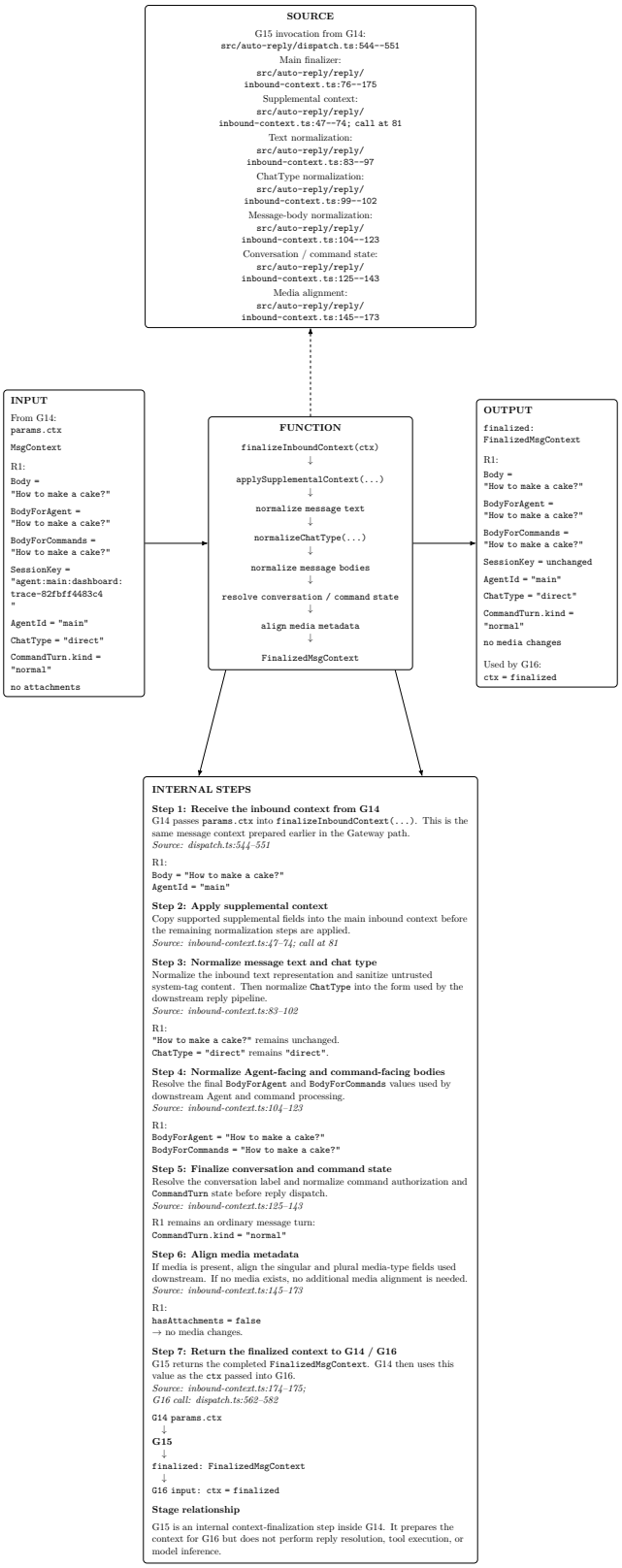
Align media-type metadata	Source
`if media paths / media metadata exist:`	src/auto-reply/reply/ inbound-context.ts:145--173
`  align MediaType and MediaTypes`	inbound-context.ts:145--173
`  pad or derive media-type entries when required`	inbound-context.ts:145--173

Return finalized context	Source
`return finalized as FinalizedMsgContext`	src/auto-reply/reply/ inbound-context.ts:174--175
`# G14 then passes finalized into dispatchReplyFromConfig (...)`	src/auto-reply/ dispatch.ts:562--582

R1 concrete state	Evidence
`R1 G15 input Body = "How to make a cake?"`	Observed: G13runtimevaluewithsourcecontext
`R1 G15 input BodyForAgent = "How to make a cake?"`	SourceDerived:G13source-derivedvalue
`R1 G15 input BodyForCommands = "How to make a cake?"`	SourceDerived:G13source-derivedvalue
`R1: SessionKey remains the canonical Session key`	SourceDerived
`R1: AgentId remains "main"`	SourceDerived
`R1: ChatType remains "direct"`	SourceDerived
`R1: command turn remains an ordinary message turn`	SourceDerived
`R1: no media-type alignment is required`	SourceDerived:noattachments
`R1: event = "inbound_context_finalized"`	Observed:3events;result=finalized;runtimeObserved=true

G15: Inbound Context Finalization Flow Diagram

Purpose: This step finalizes the inbound `MsgContext` before reply dispatch. G15 normalizes message text, chat type, message bodies, conversation metadata, command state, and media metadata, then returns a `FinalizedMsgContext` for G16.



G16: Reply Dispatch Orchestration

Purpose: This step is the main configuration-aware reply-dispatch orchestration stage. G16 receives the `FinalizedMsgContext` produced by G15 and prepares the complete downstream reply environment before and after the Agent-runtime handoff.

The function first checks whether dispatch has already been aborted, derives diagnostic and Session-lifecycle state, resolves the relevant Session-store entry, and determines the effective dispatch Session, including any ACP-bound Session. It then re-resolves the effective downstream Agent in G17, establishes reply routing and runtime state, and acquires reply-operation lifecycle ownership.

Before the default Agent reply resolver is reached, G16 loads runtime plugins, builds hook and inbound-claim contexts, conditionally stages remote media for hook consumers, resolves reply routing, evaluates the downstream send policy and effective tool policy, determines source-reply visibility, and performs physical inbound-message dedupe. Plugin-owned claims, fast-abort handling, `before_dispatch` hooks, and `reply_dispatch` hooks may terminate the path before the default Agent runtime is entered.

If none of those paths handles the message, G16 acquires final dispatch ownership and prepares the callback / delivery contract. It then reaches G18, where the supplied or default reply resolver is selected and the message is handed to the deeper reply / Agent runtime. After G18 returns, G16 resumes control, applies Session metadata changes, processes final reply payloads, performs delivery and dedupe bookkeeping, completes the diagnostic lifecycle and reply operation, and returns a `DispatchFromConfigResult`.

For R1, G16 entry is directly OBSERVED through `reply_dispatch_orchestration_entered`, with `result=entered` and `runtimeObserved=true`. The complete final `DispatchFromConfigResult` is not serialized as a standalone G16 return event. Later return edges and individual result fields remain source-derived or not directly recorded. The same execution produces the G17 event `effective_agent_reresolved` with `agentId = "main"` and `downstreamAgentId = "main"`, followed by the G18 event `reply_resolver_selected` with `resolverSource = "default_getReplyFromConfig"`. These runtime observations show that R1 traverses the normal G16 path through effective-Agent re-resolution and reaches the default reply-resolver boundary.

Python-style pseudocode	Exact source / evidence
Main G16 function and early-abort guard	**Source**
`async def dispatchReplyFromConfig(params):`	src/auto-reply/reply/dispatch-from-config.ts:1215--1217
`  ctx, cfg, dispatcher = params.ctx, params.cfg, params.dispatcher`	dispatch-from-config.ts:1218
`  if params.replyOptions.abortSignal.aborted:`	dispatch-from-config.ts:1219
`    return {"queuedFinal": False,`	dispatch-from-config.ts:1220--1221
`      "counts": dispatcher.getQueuedCounts()}`	dispatch-from-config.ts:1222--1223
Establish diagnostics and initial Session state	**Source**
`  diagnostics_enabled = isDiagnosticsEnabled(cfg)`	src/auto-reply/reply/dispatch-from-config.ts:1225
`  channel = normalize(ctx.Surface or ctx.Provider)`	dispatch-from-config.ts:1226
`  chat_id = ctx.To or ctx.From`	dispatch-from-config.ts:1227
`  message_id = ctx.MessageSid or fallback message IDs`	dispatch-from-config.ts:1228

```
   session_key = ctx.SessionKey or
ctx.CommandTargetSessionKey                                  dispatch-from-config.ts:1229--1230
   initial_session =
resolveSessionStoreLookup(ctx, cfg)                          dispatch-from-config.ts:1233
def resolveSessionStoreLookup(ctx, cfg):
                                                             src/auto-reply/reply/
                                                             dispatch-from-config.ts:441--470
   target =
resolveCommandTurnTargetSessionKey(ctx)                      dispatch-from-config.ts:450
   agent_id = resolveSessionAgentId(...)
                                                             dispatch-from-config.ts:455
   store_path = resolveStorePath(...)
                                                             dispatch-from-config.ts:456
   store = loadSessionStore(store_path)
                                                             dispatch-from-config.ts:458
```

Create diagnostic message lifecycle	Source

```
   lifecycle_session_id =
                                                             src/auto-reply/reply/
                                                             dispatch-from-config.ts:1242--1257
      initial_session.entry.sessionId
                                                             dispatch-from-config.ts:1242--1257
      if initial_session.sessionKey == session_key else
↪ None                                                       dispatch-from-config.ts:1242--1257
   message_lifecycle = createDiagnosticMessageLifecycle
↪ (...)                                                      dispatch-from-config.ts:1246--1257
   trace_attributes = {surface, hasSessionKey, hasRunId}
                                                             dispatch-from-config.ts:1258--1262
   traceReplyPhase(name, run) =
                                                             dispatch-from-config.ts:1266--1273
      measureDiagnosticsTimelineSpan(name, run, phase="
↪ agent-turn")                                               dispatch-from-config.ts:1268--1272
```

Resolve effective dispatch Session	Source

```
   bound_acp_session = resolveBoundAcpDispatchSessionKey(
                                                             src/auto-reply/reply/
                                                             dispatch-from-config.ts:1341
      ctx=ctx, cfg=cfg
                                                             dispatch-from-config.ts:1341
   )
                                                             dispatch-from-config.ts:1341
def resolveBoundAcpDispatchSessionKey(params):
                                                             src/auto-reply/reply/
                                                             dispatch-from-config.ts:473--502
   acp_dispatch_session_key =
                                                             src/auto-reply/reply/
                                                             dispatch-from-config.ts:1342--1343
      bound_acp_session or initial_session.sessionKey or
↪ session_key                                                dispatch-from-config.ts:1342--1343
   dispatch_operation_session_key =
                                                             dispatch-from-config.ts:1347--1348
      initial_session.sessionKey or session_key or
↪ acp_dispatch_session_key                                   dispatch-from-config.ts:1347--1348
```

Invoke G17: effective Agent re-resolution	Source

```
   session_agent_id = resolveSessionAgentId(
                                                             src/auto-reply/reply/
                                                             dispatch-from-config.ts:1422--1427
```

```
      sessionKey=acp_dispatch_session_key,
                                                    dispatch-from-config.ts:1422--1427
      config=cfg,
                                                    dispatch-from-config.ts:1422--1427
      fallbackAgentId=ctx.AgentId
                                                    dispatch-from-config.ts:1422--1427
   )
                                                    dispatch-from-config.ts:1422--1427
   session_agent_cfg = resolveAgentConfig(cfg,
↪ session_agent_id)                                 dispatch-from-config.ts:1422--1427
R1: session_agent_id = "main"
                                                    Observed:TraceClawG17runtimeevent
R1: downstream_agent_id = "main"
                                                    Observed:TraceClawG17runtimeevent
```

Establish Agent / reply environment	Source

```
   verbose_progress = createShouldEmitVerboseProgress
↪ (...)                                             src/auto-reply/reply/
                                                    dispatch-from-config.ts:1428--1439
   reply_route = resolveEffectiveReplyRoute(
                                                    dispatch-from-config.ts:1442
      ctx=ctx, entry=session_store_entry.entry
                                                    dispatch-from-config.ts:1442
   )
                                                    dispatch-from-config.ts:1442
   route_thread_id = resolveRoutedDeliveryThreadId(...)
                                                    dispatch-from-config.ts:1447--1450
```

Establish reply-operation lifecycle admission	Source

```
async def runWithDispatchLifecycleAdmission(run):
                                                    src/auto-reply/reply/
                                                    dispatch-from-config.ts:1521--1528
   if dispatch_reply_operation:
                                                    dispatch-from-config.ts:1522--1524
      return runWithReplyOperationLifecycleAdmission(...)
                                                    dispatch-from-config.ts:1522--1524
   else use pre-dispatch lifecycle admission or run
↪ directly                                          dispatch-from-config.ts:1525--1527
async def ensureDispatchReplyOperation(phase):
                                                    src/auto-reply/reply/
                                                    dispatch-from-config.ts:1533--1623
```

Recover stale work and admit the reply turn	Source

```
   should_recover_stale_visible_operation = ...
                                                    src/auto-reply/reply/
                                                    dispatch-from-config.ts:1593--1598
   recovery = requestStuckDiagnosticSessionRecovery(...)
                                                    dispatch-from-config.ts:1599--1608
   admission = await admitReplyTurn(
                                                    dispatch-from-config.ts:1609--1623
      sessionKey=dispatch_operation_session_key,
                                                    dispatch-from-config.ts:1610
      sessionId=operation_session_id,
                                                    dispatch-from-config.ts:1611
      expectedSessionId=resolveOperationExpectedSessionId
↪ (),                                               dispatch-from-config.ts:1612
      kind=reply_turn_kind,
                                                    dispatch-from-config.ts:1615
      upstreamAbortSignal=params.replyOptions.abortSignal
                                                    dispatch-from-config.ts:1618
```

```
)
```
dispatch-from-config.ts:1609--1623

Load runtime plugins and construct hook context	Source
`ensure_runtime_plugins = await traceReplyPhase(`	src/auto-reply/reply/
	dispatch-from-config.ts:1877--1879
`"reply.load_runtime_plugins", loadRuntimePlugins`	dispatch-from-config.ts:1877--1879
`)`	dispatch-from-config.ts:1877--1879
`await traceReplyPhase("reply.ensure_runtime_plugins", ...)`	dispatch-from-config.ts:1880--1882
`hook_runner = getGlobalHookRunner()`	dispatch-from-config.ts:1883
`hook_context, inbound_claim_context, inbound_claim_event =`	dispatch-from-config.ts:1889--1904
`buildHookState(ctx)`	dispatch-from-config.ts:1889--1904

Conditionally stage remote media for hooks	Source
`async def prepareHookMediaMetadata():`	src/auto-reply/reply/
	dispatch-from-config.ts:1916--1928
`staged = await traceReplyPhase(`	dispatch-from-config.ts:1916--1924
`"reply.stage_remote_media_for_dispatch",`	dispatch-from-config.ts:1916--1924
`stageRemoteInboundMediaIfNeeded(...)`	dispatch-from-config.ts:1917--1923
`)`	dispatch-from-config.ts:1916--1924
`R1: no attachments`	Observed:nativeruntimevalue;SourceDerivedbranchexclusion

Resolve effective reply-routing behavior	Source
`normalize Provider / Surface / reply route`	src/auto-reply/reply/
	dispatch-from-config.ts:1998--2014
`route_decision = resolveReplyRoutingDecision(`	dispatch-from-config.ts:2003--2011
`provider=ctx.Provider,`	dispatch-from-config.ts:2004
`surface=ctx.Surface,`	dispatch-from-config.ts:2005
`originatingChannel=reply_route.channel,`	dispatch-from-config.ts:2007
`originatingTo=reply_route.to`	dispatch-from-config.ts:2008
`)`	dispatch-from-config.ts:2003--2011
`delivery_channel = originating channel if routed else current surface`	dispatch-from-config.ts:2012--2014

Re-evaluate downstream send policy	Source
`send_policy = resolveSendPolicy(`	src/auto-reply/reply/
	dispatch-from-config.ts:2184--2196
`cfg=cfg,`	dispatch-from-config.ts:2184--2196

```
    entry=session_store_entry.entry,                    dispatch-from-config.ts:2186
    sessionKey=session_store_entry.sessionKey or        dispatch-from-config.ts:2187
↪ session_key,
    channel=effective delivery channel,                 dispatch-from-config.ts:2188--2194
    chatType=session_store_entry.entry.chatType         dispatch-from-config.ts:2195
)                                                       dispatch-from-config.ts:2184--2196
```

Resolve effective tool and source-reply policies — Source

```
tool_policy = resolveEffectiveToolPolicy(           src/auto-reply/reply/
                                                    dispatch-from-config.ts:2206--2210
    config=cfg,                                     dispatch-from-config.ts:2207
    sessionKey=acp_dispatch_session_key,            dispatch-from-config.ts:2208
    agentId=session_agent_id                        dispatch-from-config.ts:2209
)                                                   dispatch-from-config.ts:2206--2210
message_tool_available = isToolAllowedByPolicies(   src/auto-reply/reply/
                                                    dispatch-from-config.ts:2289--2299
    "message", effective policy layers              dispatch-from-config.ts:2289--2299
)                                                   dispatch-from-config.ts:2289--2299
source_reply_policy =                               src/auto-reply/reply/
↪ resolveSourceReplyVisibilityPolicy(...)           dispatch-from-config.ts:2300--2321
derive suppression / visibility state               dispatch-from-config.ts:2300--2321
```

Claim physical inbound-message dedupe ownership — Source

```
inbound_dedupe_claim = claimInboundDedupe(ctx)      src/auto-reply/reply/
                                                    dispatch-from-config.ts:2342
if status in ["duplicate", "inflight"]:             dispatch-from-config.ts:2343--2349
    recordProcessed("skipped", reason="duplicate")  dispatch-from-config.ts:2344
    return without new dispatch                     dispatch-from-config.ts:2345--2348
define commit / release helpers for claimed dedupe  dispatch-from-config.ts:2350--2359
```

Run message-received and fast-abort paths — Source

```
def emitMessageReceivedHooks():                     src/auto-reply/reply/
                                                    dispatch-from-config.ts:2399--2425
run plugin message_received hooks when enabled      dispatch-from-config.ts:2400--2411
run internal message:received hook when Session key dispatch-from-config.ts:2413--2424
↪ exists
markProcessing()                                    dispatch-from-config.ts:2426
load / execute fast-abort resolver                  dispatch-from-config.ts:2428--2474
```

```
if fast_abort.handled: return early result
```
dispatch-from-config.ts:2436--2474

Acquire pre-dispatch reply ownership	Source
`pre_dispatch = await ensureDispatchReplyOperation("pre_dispatch")`	src/auto-reply/reply/ dispatch-from-config.ts:2478
`if pre_dispatch.status == "aborted": return aborted result`	dispatch-from-config.ts:2479--2480
`if pre_dispatch.status == "busy": return busy result`	dispatch-from-config.ts:2482--2483
`if plugin-owned binding exists: evaluate plugin ownership path`	dispatch-from-config.ts:2486--2633

Allow plugin interception before Agent dispatch	Source
`if hook_runner.hasHooks("before_dispatch"):`	src/auto-reply/reply/ dispatch-from-config.ts:2908--2968
`before_dispatch_result = await traceReplyPhase(`	dispatch-from-config.ts:2910--2947
`"reply.before_dispatch_hooks", ...`	dispatch-from-config.ts:2910--2947
`)`	dispatch-from-config.ts:2910--2947
`if handled: deliver result and return`	dispatch-from-config.ts:2948--2968
`if hook_runner.hasHooks("reply_dispatch"):`	src/auto-reply/reply/ dispatch-from-config.ts:2973--3023
`reply_dispatch_result = await traceReplyPhase(`	dispatch-from-config.ts:2974--3014
`"reply.reply_dispatch_hooks", ...`	dispatch-from-config.ts:2974--3014
`)`	dispatch-from-config.ts:2974--3014
`if handled: return hook result`	dispatch-from-config.ts:3015--3023

Acquire final dispatch ownership and prepare callbacks	Source
`dispatch_acquisition = await ensureDispatchReplyOperation("dispatch")`	src/auto-reply/reply/ dispatch-from-config.ts:3025
`if aborted: return aborted result`	dispatch-from-config.ts:3026--3027
`if busy: return busy result`	dispatch-from-config.ts:3029--3030
`wrapProgressCallback gates forwarding; onItemEvent handles progress`	src/auto-reply/reply/ dispatch-from-config.ts: 3297--3326,3357--3371

Invoke G18: reply-resolver boundary	Source
`reply_resolver = params.replyResolver or`	src/auto-reply/reply/ dispatch-from-config.ts:3381--3384
`traceReplyPhase("reply.load_reply_resolver",`	dispatch-from-config.ts:3381--3384
`loadGetReplyFromConfigRuntime)`	dispatch-from-config.ts:3381--3384

```
   reply_config = withFullRuntimeReplyConfig(...)
                                                          dispatch-from-config.ts:3385--3387
   recordAgentDispatchStarted()
                                                          dispatch-from-config.ts:3388
   reply_result = await
runWithDispatchLifecycleAdmission(                        dispatch-from-config.ts:3389--3811
      traceReplyPhase("reply.run_reply_resolver",
                                                          dispatch-from-config.ts:3389--3811
         reply_resolver(ctx, options,
reply_config)                                             dispatch-from-config.ts:3389--3811
      )
                                                          dispatch-from-config.ts:3389--3811
   )
                                                          dispatch-from-config.ts:3389--3811
R1: resolver_source = "default_getReplyFromConfig"
                                                          Observed:TraceClawG18runtimeevent
```

Resume G16 after G18 returns	Source
`session_metadata_changes = takeCommandSessionMetadataChanges(ctx)`	src/auto-reply/reply/dispatch-from-config.ts:3812
`notifySessionMetadataChanges(session_metadata_changes)`	dispatch-from-config.ts:3813
`final_dispatch = await ensureDispatchReplyOperation("dispatch")`	dispatch-from-config.ts:3814
`if ACP tail dispatch is required: process special tail path`	dispatch-from-config.ts:3827--3884

Normalize and deliver final reply payloads	Source
`replies = normalize reply_result into a list`	src/auto-reply/reply/dispatch-from-config.ts:3886
`flushPendingCommentaryProgress()`	dispatch-from-config.ts:3889
`for reply in replies:`	dispatch-from-config.ts:3910
`apply reasoning / commentary / suppression checks`	dispatch-from-config.ts:3914--3936
`dedupe final payload`	dispatch-from-config.ts:3937--3941
`final_reply = await sendFinalPayload(reply, ...)`	dispatch-from-config.ts:3943
`optionally perform TTS-only final delivery`	dispatch-from-config.ts:3963--4038

Complete successful G16 dispatch	Source
`await waitForPendingDirectBlockReplyDelivery(...)`	src/auto-reply/reply/dispatch-from-config.ts:4040
`counts = dispatcher.getQueuedCounts()`	dispatch-from-config.ts:4041
`counts.final += routed_final_count`	dispatch-from-config.ts:4042
`commitInboundDedupeIfClaimed()`	dispatch-from-config.ts:4043
`recordAgentDispatchCompleted("completed")`	dispatch-from-config.ts:4044
`recordProcessed("completed", ...)`	dispatch-from-config.ts:4045--4048
`markIdle("message_completed")`	dispatch-from-config.ts:4049

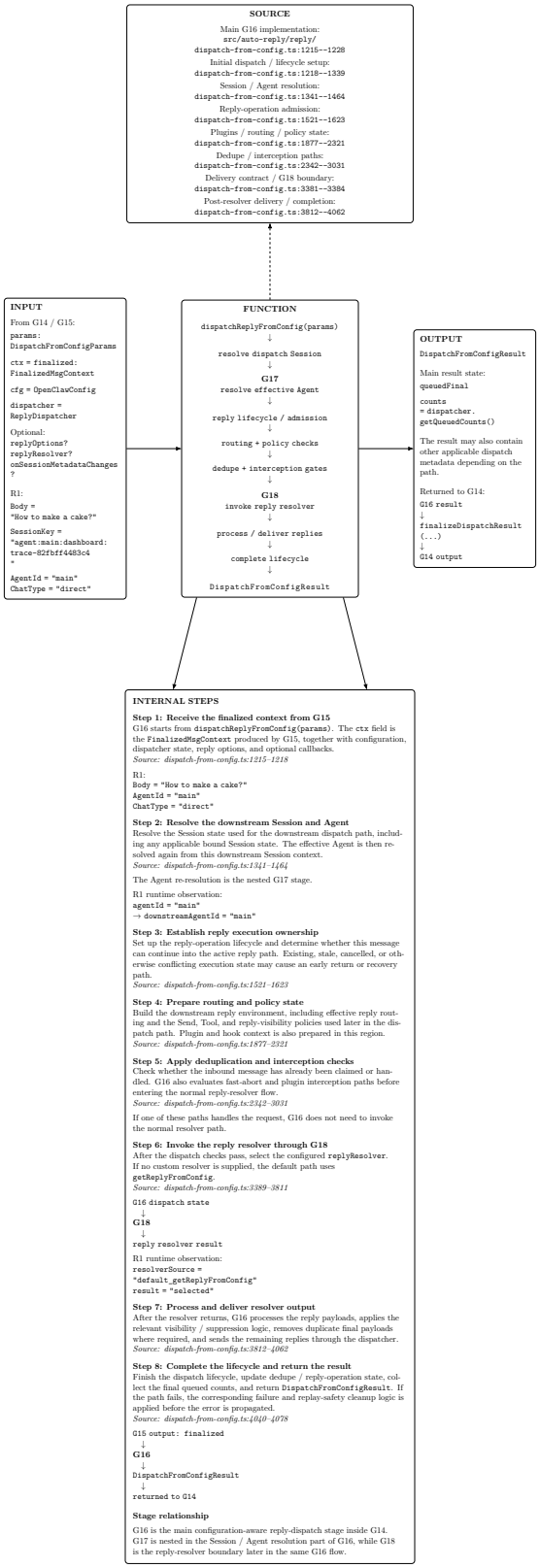
`completeDispatchReplyOperation()`	dispatch-from-config.ts:4050
`return DispatchFromConfigResult(...)`	dispatch-from-config.ts:4051--4062

Error and replay-safety path	Source
`except error:`	src/auto-reply/reply/ dispatch-from-config.ts:4063--4078
`if operation aborted: return aborted result`	dispatch-from-config.ts:4064--4066
`commit or release inbound dedupe according to replay safety`	dispatch-from-config.ts:4067--4073
`recordAgentDispatchCompleted("error", ...)`	dispatch-from-config.ts:4074
`recordProcessed("error", ...)`	dispatch-from-config.ts:4075
`markIdle("message_error")`	dispatch-from-config.ts:4076
`failDispatchReplyOperation(error)`	dispatch-from-config.ts:4077
`raise error`	dispatch-from-config.ts:4078

R1 concrete state	Evidence
`R1: message = "How to make a cake?"`	SourceDerived:G13/G15source-derivedvalue
`R1: session_key = "agent:main:dashboard:trace-82fbff4483c4"`	Observed Observed
`R1: upstream AgentId = "main"`	Observed
`R1: G17 downstreamAgentId = "main"`	Observed:TraceClawG17runtimeevent
`R1: G17 result = "resolved"`	Observed:TraceClawG17runtimeevent
`R1: G18 resolverSource = "default_getReplyFromConfig"`	Observed:TraceClawG18runtimeevent Observed:TraceClawG18runtimeevent
`R1: G18 result = "selected"`	Observed:TraceClawG18runtimeevent
`R1: event = "reply_dispatch_orchestration_entered"`	Observed: result=entered;runtimeObserved=true
`R1: exact G16 final counts are not asserted here`	NotObserved:notdirectlyobserved

G16: Reply Dispatch Orchestration Flow Diagram

Purpose: This step receives the finalized context from G15 and carries out the configuration-aware reply-dispatch flow. G16 resolves the downstream Session and Agent, establishes reply execution state, applies routing and policy checks, handles deduplication and interception paths, invokes the reply resolver through G18, and returns the resulting `DispatchFromConfigResult` to G14.



G17: Downstream Effective Agent Re-resolution

Purpose: This step re-resolves the effective Agent inside the G16 reply-dispatch path. Instead of relying only on the upstream `ctx.AgentId`, G17 uses the downstream dispatch Session key together with the current configuration to determine which Agent should be used for the remaining reply path. The resolved Agent configuration is then loaded and used by later G16 processing.

G17 is a nested step inside G16 rather than a separate Gateway request stage. Its main

purpose is to keep the downstream Agent identity consistent with the Session that G16 is actually using for reply dispatch.

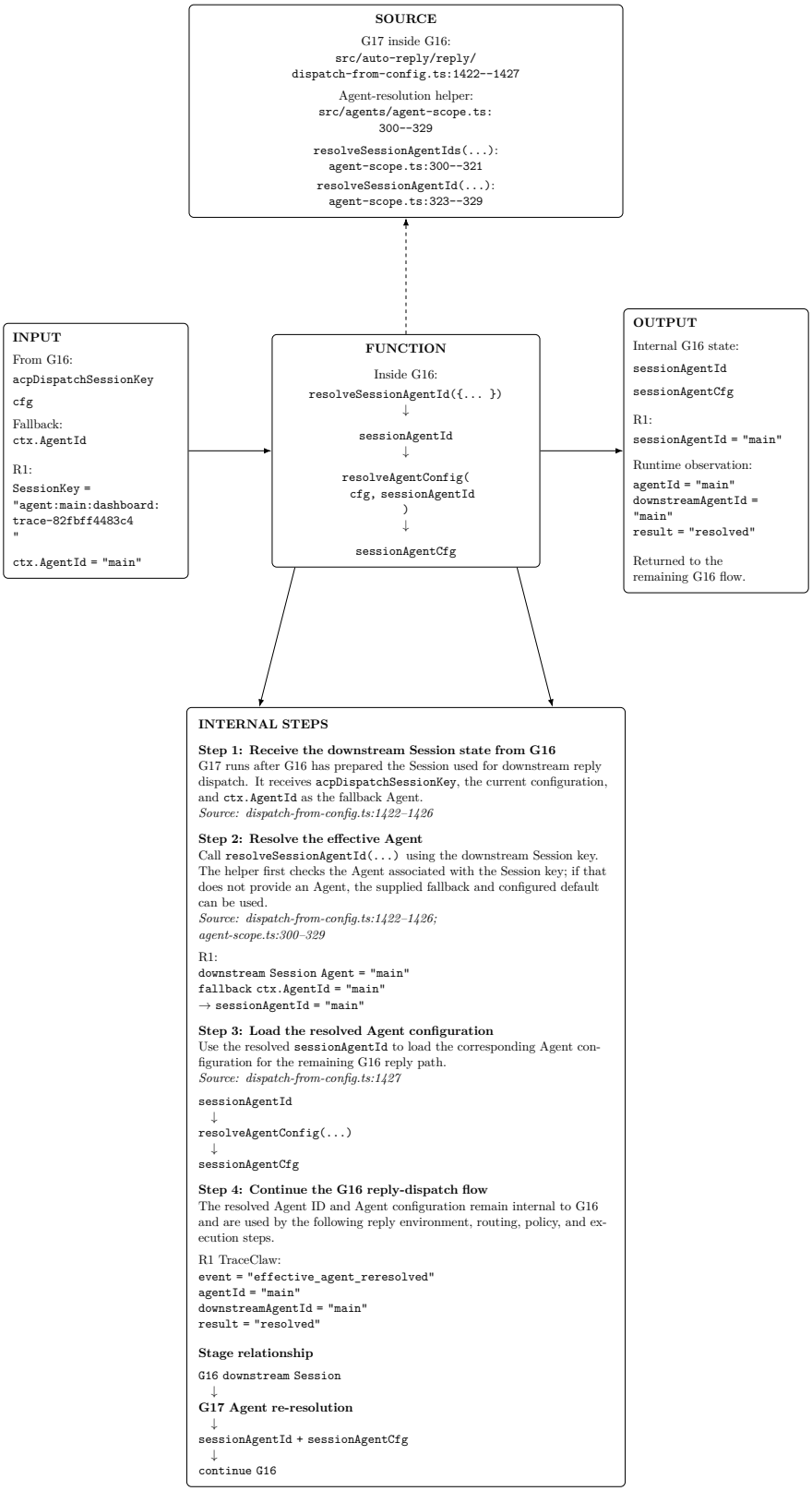
The source performs two operations. First, `resolveSessionAgentId(...)` is called with `acpDispatchSessionKey`, the current configuration, and `ctx.AgentId` as the fallback Agent. The resulting `sessionAgentId` is then passed to `resolveAgentConfig(...)` to obtain the corresponding Agent configuration.

The underlying Agent-resolution helper gives priority to an explicit Agent ID when one is supplied, then to the Agent encoded in the Session key, followed by the fallback Agent and finally the configured default Agent. In the G17 call, no separate explicit `agentId` argument is supplied, so the downstream Session key is checked before `ctx.AgentId` is used as fallback.

For R1, the downstream Session still identifies Agent `"main"`, and the fallback `ctx.AgentId` is also `"main"`. TraceClaw therefore records `agentId = "main"` and `downstreamAgentId = "main"`, with `result = "resolved"`.

G17: Downstream Effective Agent Re-resolution Flow Diagram

Purpose: This step re-resolves the effective Agent from the downstream Session used by G16. The resolved Agent ID and its configuration are then returned to the surrounding G16 reply-dispatch flow.



G18: Reply Resolver Boundary

Purpose: This step selects the reply resolver used by the current G16 dispatch path and invokes it with the finalized message context, reply options, and runtime configuration. If no custom resolver is supplied, OpenClaw loads the default `getReplyFromConfig` implementation.

G18 is the boundary between the Gateway-side reply orchestration in G16 and the deeper reply / Agent runtime. The resolver runs under the existing dispatch lifecycle and abort controls. When it returns, the result is handed back to G16 for the remaining reply-processing and delivery

steps.

For the normal path, G18 first checks `params.replyResolver`. If no custom resolver is present, `loadGetReplyFromConfigRuntime()` loads the default runtime and selects `getReplyFromConfig`. G18 then prepares the effective reply configuration, records the start of Agent dispatch, and invokes the selected resolver.

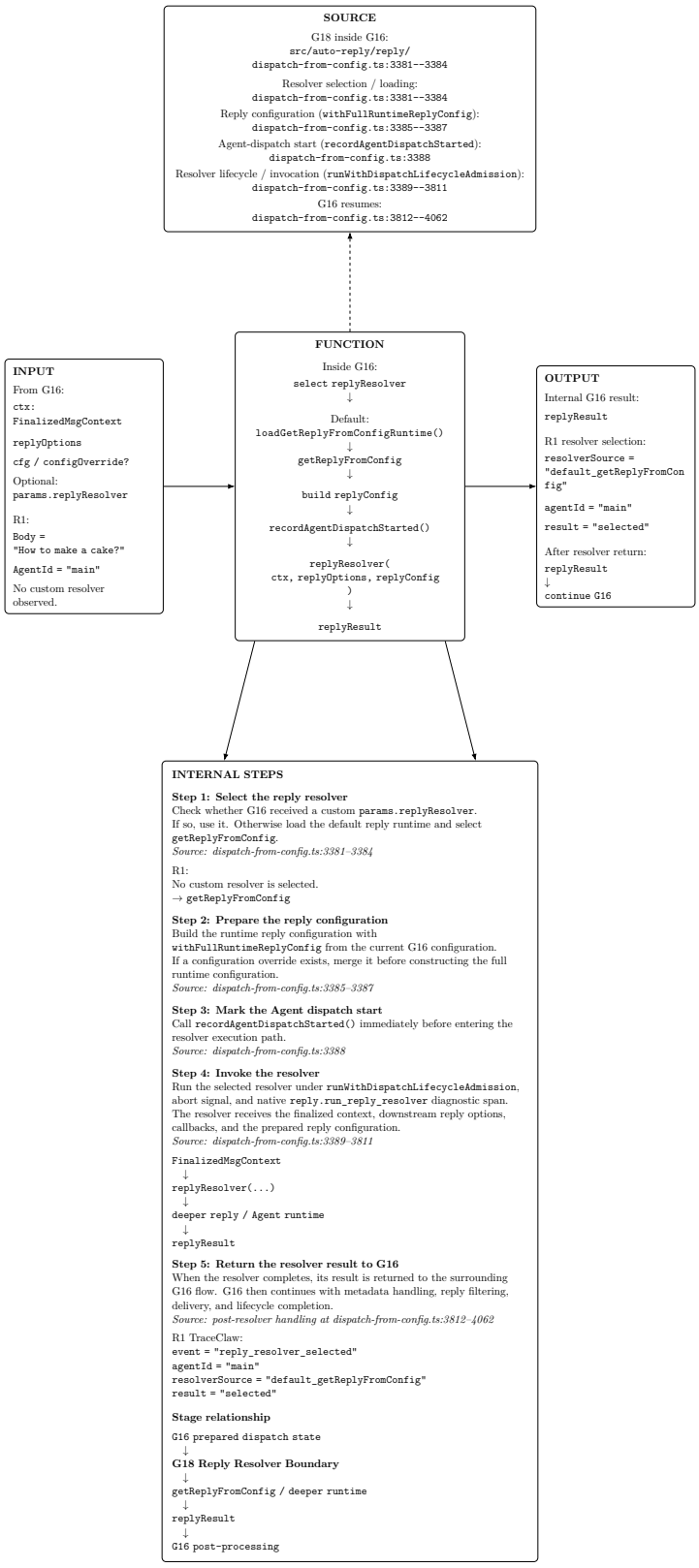
For R1, no custom reply resolver was selected. TraceClaw records `resolverSource="default_getReplyFromConfig"` and `result = "selected"`. The effective Agent entering this boundary remains `"main"`.

Python-style pseudocode	Exact source / evidence
## G18 begins inside dispatchReplyFromConfig	src/auto-reply/reply/dispatch-from-config.ts:3381--3384
reply_resolver = params.replyResolver	dispatch-from-config.ts:3381--3384
if reply_resolver is None:	dispatch-from-config.ts:3381--3384
runtime = await loadGetReplyFromConfigRuntime()	dispatch-from-config.ts:3381--3384
reply_resolver = runtime.getReplyFromConfig	dispatch-from-config.ts:3381--3384
reply_config = withFullRuntimeReplyConfig(...)	dispatch-from-config.ts:3385--3387
# optional configOverride is merged before use	dispatch-from-config.ts:3385--3387
recordAgentDispatchStarted()	dispatch-from-config.ts:3388
reply_result = await runWithDispatchLifecycleAdmission(...)	dispatch-from-config.ts:3389--3811
runWithDispatchAbortSignal(...)	dispatch-from-config.ts:3389--3811
traceReplyPhase("reply.run_reply_resolver", ...)	dispatch-from-config.ts:3389--3811
reply_resolver(	dispatch-from-config.ts:3389--3811
ctx,	dispatch-from-config.ts:3389--3811
reply_options,	dispatch-from-config.ts:3389--3811
reply_config	dispatch-from-config.ts:3389--3811
)	dispatch-from-config.ts:3389--3811
# reply_result returns to the surrounding G16 flow	dispatch-from-config.ts:3812--4062
## R1 runtime	Observed:TraceClawruntimeevent
agent_id = "main"	Observed:G18reply_resolver_selected
resolver_source = "default_getReplyFromConfig"	Observed:G18reply_resolver_selected
result = "selected"	Observed:G18reply_resolver_selected

G18: Reply Resolver Boundary Flow Diagram

Purpose: This step selects and invokes the reply resolver used by G16. The default path loads `getReplyFromConfig`, prepares the runtime reply configuration, and crosses into the deeper

reply / Agent runtime. The returned reply result is then handed back to G16.



OpenClaw Gateway Flow: G0–G18

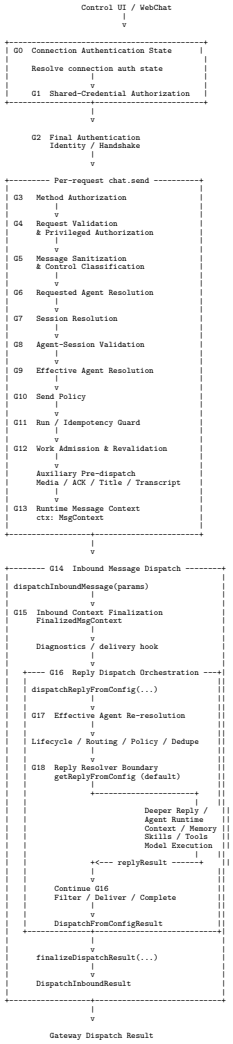


Figure 1: Full OpenClaw Gateway flow from Control UI / WebChat through G0–G18 and the return from the deeper reply / Agent Runtime. This figure is kept in the appendix because it is intended as an audit map rather than a compact main-paper framework.

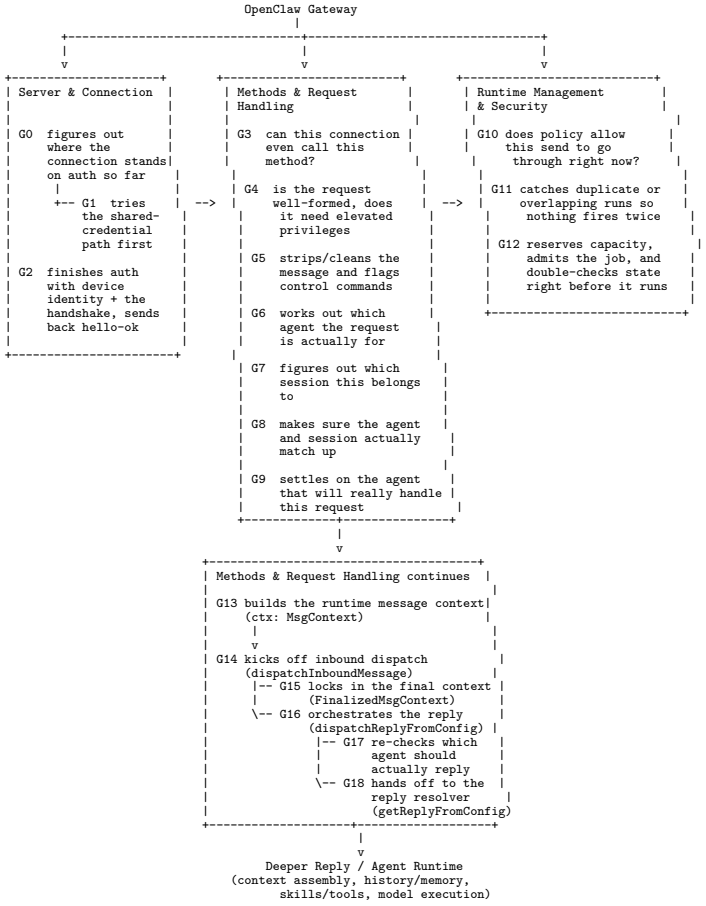


Figure 2: G0–G18 Gateway stages, grouped into three modules by where they live in the source: Server & Connection (G0–G2), Methods & Request Handling (G3–G9, G13–G18), and Runtime Management & Security (G10–G12). Each label is a quick gloss of what that stage actually does, not a formal spec.

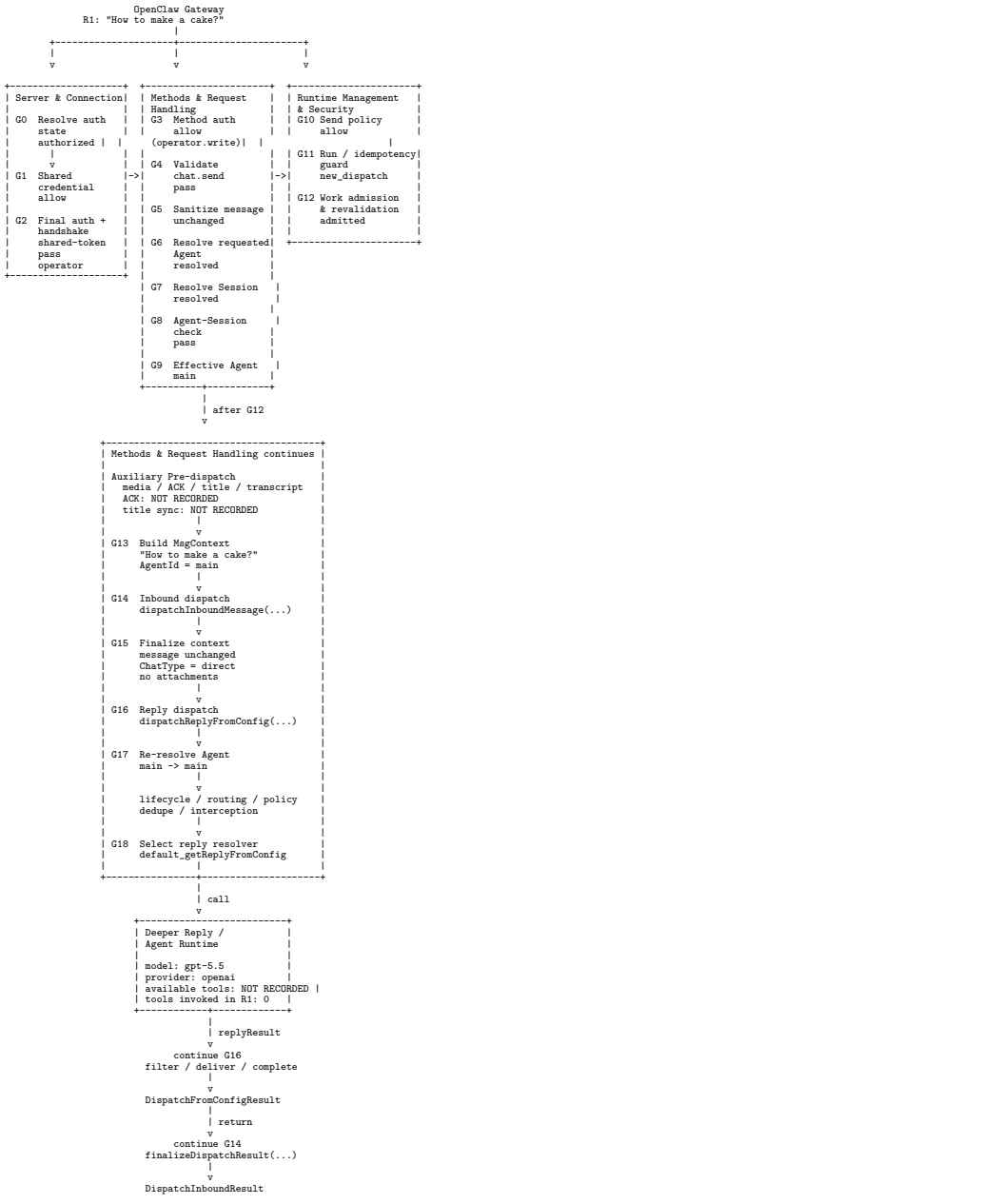


Figure 3: OpenClaw Gateway G0–G18 flow using R1 (`"How to make a cake?"`). The stages are grouped into Server & Connection, Methods & Request Handling, and Runtime Management & Security. Values shown beside the stages are the main results observed in R1.

2 Post-G18 Agent Runtime Source and Runtime Evidence

The Gateway stage model stops at G18 by design. After G18 selects the reply resolver, OpenClaw enters the deeper Agent Runtime. TraceClaw keeps that path in a separate `agentRuntime` object rather than adding a synthetic G19. This section records the source path, event vocabulary, and the evidence boundary used in the paper.

A. Verified Post-G18 Source Path

```
G18 replyResolver(...)
  -> getReplyFromConfig(...)
  -> runPreparedReply(...)
  -> runReplyAgent(...)
  -> runAgentTurnWithFallback(...)
       -> CLI branch      -> runCliAgentWithLifecycle(...)
       -> embedded branch -> runEmbeddedAgent(...)
              -> resolve Skills prompt
              -> construct runtime Tools
              -> create Agent session
              -> subscribe Agent lifecycle/tool events
              -> activeSession.prompt(...)
              -> assistant/tool event loop
              -> winning EmbeddedAgentRunResult
              -> finalAssistantVisibleText / finalAssistantRawText
              -> agent_reply_finalized
              -> deferred Agent terminal lifecycle
  -> replyResult returns across G18
  -> G16 resumes post-processing / delivery / completion
  -> G14 returns final DispatchInboundResult
```

The final answer is taken from the winning embedded run result and the later resolver return, not from an earlier message-end hook.

B. Source Anchors

Boundary	Fixed OpenClaw v2026.7.1-2 source anchor
G18 resolver selection / invocation	src/auto-reply/reply/dispatch-from-config.ts:3381--3384
Default resolver	src/auto-reply/reply/get-reply.ts
Prepared reply runner	src/auto-reply/reply/get-reply-run.ts:512--1641
Reply Agent runner	src/auto-reply/reply/agent-runner.ts:1130--2753
CLI / embedded runtime selection	src/auto-reply/reply/agent-runner-execution.ts:2338--2660
Skills prompt construction	src/agents/embedded-agent-runner/run/attempt.ts:1150--1158
Runtime tool construction	src/agents/embedded-agent-runner/run/attempt.ts:1342--1383
Agent session creation	src/agents/embedded-agent-runner/run/attempt.ts:2557--2573
Model prompt boundary	src/agents/embedded-agent-runner/run/attempt.ts:3582--3588
Agent event subscription	src/agents/embedded-agent-runner/run/attempt.ts:3711--3749
Agent lifecycle start / end	src/agents/embedded-agent-subscribe.handlers.lifecycle.ts:27--58,170--225
Tool start / result	src/agents/embedded-agent-subscribe.handlers.tools.ts:1049--1059,1489--1502
Final embedded winner result	src/agents/embedded-agent-runner/run.ts
TraceClaw final-reply capture	src/auto-reply/reply/agent-runner-execution.ts winning fallbackResult.result boundary
Global Agent event bus	src/infra/agent-events.ts:487--492

C. Runtime Event Vocabulary

Event	Meaning	Evidence role
`agent_runtime_selected`	Records the effective Agent Runtime attempt, including Agent, runner, provider, and model when available.	Runtime handoff evidence.
`agent_run_started`	Records the Agent lifecycle start for the correlated run.	Runtime entry evidence.
`tool_started` / `tool_result`	Records tool calls and sanitized results inside the deeper runtime.	Tool-loop evidence.
`agent_reply_finalized`	Records final reply text from the winning run result.	Final-reply evidence.
`agent_run_ended`	Records terminal Agent lifecycle state and stop reason.	Lifecycle completion evidence.
`reply_resolver_returned`	Records resolver source, reply-result kind, and reply count when control returns from the resolver to `dispatchFromConfig`. Later G16/G14 post-processing is source-derived unless separately instrumented.	Resolver-return evidence.

D. Controlled AR-Layer Ledgers

The controlled cohort uses the same AR vocabulary as the Gateway ledger. AR0–AR5 are runtime-observed when the saved trace contains the selected runtime, run start, tool/no-tool outcome, finalized reply, and run end. AR6 is treated separately: `reply_resolver_returned` observes the resolver return event, while the later G16/G14 resume is source-derived unless a separate post-return event is present.

R1 no-tool path. R1 is the detailed Gateway case. Its post-G18 path contains an embedded Agent run with no tool calls. The trace records `agent_runtime_selected`, `agent_run_started`, `agent_reply_finalized`, `agent_run_ended`, and `reply_resolver_returned`. The absence of a tool event is part of the evidence: AR2 is `Observed(no_tool)`, not missing.

AR stage	R1 evidence	R1 value	Status
AR0	Runtime selected	runner `embedded`; Agent `main`; provider `openai`; model `gpt-5.5`	Observed
AR1	Agent run started	run is correlated with R1 run ID and Session key	Observed
AR2	Tool loop	no `tool_started` / `tool_result` pair is present	Observed
AR3	Final reply	final reply event present; text redacted from the audit table	Observed
AR4	Run ended	terminal lifecycle event has stop reason `stop`	Observed
AR5	Resolver returned	`reply_resolver_returned`; reply-result kind `single`	Observed
AR6	Return to Gateway post-processing	G16/G14 continuation follows from source after resolver return	SourceDerived

R3 external side-effect path. R3 is the controlled run that exercises an external side effect through a host application. The trace contains five `bash` tool calls: four capability probes and one Apple Mail invocation through `osascript`. The four probes are `command -v osascript || true`, `command -v mutt || true`, `command -v mail || true`, and `command -v msmtp || true`. The tool results record exit status and timing, but not stdout, so the exact probe output and the reason for choosing `osascript` are rationale-level gaps. The final `osascript` command exits with code 0, which supports only host-side command completion; it does not verify downstream email delivery. No separate approval or confirmation event is observed beyond the user's prompt.

Order	Tool	Recorded evidence	Status
1	`bash`	`command -v osascript || true`; stdout not recorded	Observed
2	`bash`	`command -v mutt || true`; stdout not recorded	Observed
3	`bash`	`command -v mail || true`; stdout not recorded	Observed
4	`bash`	`command -v msmtp || true`; stdout not recorded	Observed
5	`bash`	Apple Mail invocation through `osascript`; exit code 0	Observed

Compact AR summary for R1–R7.

Run	AR observed	AR2 outcome	Tool result ratio	AR6
R1	6/7	Observed(no_tool)	n/a	SourceDerived
R2	6/7	Observed(tool_results)	6/6	SourceDerived
R3	6/7	Observed(tool_results)	5/5	SourceDerived
R4	6/7	Observed(tool_results)	4/4	SourceDerived
R5	6/7	Observed(no_tool)	n/a	SourceDerived
R6	6/7	Observed(no_tool)	n/a	SourceDerived
R7	6/7	Observed(no_tool)	n/a	SourceDerived

E. Controlled Cohort Tables

The following tables are generated from saved R1–R7 trace artifacts. They are included here to keep the main paper compact while preserving machine-checkable evidence for the controlled cohort.

Run summary.

Run	Request label	Runtime class	Observed tools	Gateway / AR
R1	cake baseline	no-tool	none	complete; observed
R2	OpenTelemetry retrieval	retrieval	`web_search` ×6	complete; observed
R3	redacted email action	external side effect	`bash` ×5	complete; observed
R4	Fibonacci file creation	local write	`bash` ×3 + `apply_patch`	complete; observed
R5	tracing/provenance explanation	no-tool	none	complete; observed
R6	tracing/provenance repeat	no-tool	none	complete; observed
R7	cake repeat	no-tool	none	complete; observed

Stage-by-run branch matrix.

Stage	Branch	Runs	Rationale visibility
G0	authorized	all R1–R7	partial
G1	allow	all R1–R7	partial
G2	pass	all R1–R7	partial
G3	allow	all R1–R7	partial
G4	pass	all R1–R7	partial
G5	unchanged	all R1–R7	partial
G6	resolved	all R1–R7	partial
G7	resolved	all R1–R7	partial
G8	pass	all R1–R7	partial
G9	resolved	all R1–R7	partial
G10	allow	all R1–R7	partial
G11	pass	all R1–R7	partial
G12	admitted	all R1–R7	partial
G13	constructed	all R1–R7	partial
G14	entered	all R1–R7	partial
G15	finalized	all R1–R7	partial
G16	normal_path	all R1–R7	partial
G17	resolved	all R1–R7	partial
G18	selected	all R1–R7	partial

Execution classes.

Run	Request label	Class	Observed tools
R1	cake baseline	no-tool	none
R2	OpenTelemetry retrieval	read-only retrieval	`web_search` ×6
R3	redacted email action	external side effect via host app	`bash` ×5
R4	Fibonacci file creation	local file write	`bash` ×3 + `apply_patch`
R5	tracing/provenance explanation	no-tool	none
R6	tracing/provenance repeat	no-tool	none
R7	cake repeat	no-tool	none

Repeatability pairs.

Pair	G0–G18 branch diff	Agent	Resolver	Tools	AR structure
R1/R7	none	yes	yes	yes	yes
R5/R6	none	yes	yes	yes	yes

F. Historical Trace Notes

Historical Cake2. The older Cake2 artifact is retained for provenance only. It came from an earlier instrumentation generation and is not used in the R1–R7 quantitative analysis. Its useful role is qualitative: it shows a previously observed path with device-token fallback and an `operator.admin` authorization state. That historical contrast is kept separate from R1, whose controlled path uses shared-token authentication, role `operator`, scope `operator.write`, and the ordinary G3 required-scope check.

Historical Weather. Historical New York and Beijing weather traces are also retained as artifact-only context. They suggest that the G0–G18 Gateway skeleton can remain stable across successful tool-using requests, but they are not the current controlled cohort. The saved New York evidence contains six near-identical Weather requests: four contain only `web_search`, and two contain `bash` setup / skill-loading activity followed by `web_search`. The Beijing trace is a separate historical example with `bash` followed by `web_search`; it is not counted as one of the six New York runs. Current quantitative claims in the paper therefore cite R1–R7, not the historical Weather runs.

G. Anonymous Artifact Handles

Item	Double-blind value
Artifact repository	`ANONYMIZED-ARTIFACT-URL`
Demo video	`ANONYMIZED-DEMO-URL`
Public author-identifying links	Withheld during review; restore only after review if allowed.

3 Experimental Settings and Artifact Checklist

This section records the R1 setting. Missing fields are marked rather than guessed.

A. System and Software Setting

Item	Recorded value	Meaning / role
Target system	OpenClaw v2026.7.1-2 (0790d9f) with a TraceClaw instrumentation patch applied	Agent system whose Gateway path is analyzed.
OpenClaw version	`v2026.7.1-2`	Version used for the source-anchored trace.
Commit	0790d9f593ad30c940ed93b5872a8cf6d6f3cf8c	Source snapshot used for line-level evidence.
Entry path	Control UI / WebChat `chat.send`	Concrete request path traced in this paper.
Trace label	R1	Name of the concrete case study.
User message	`How to make a cake?`	Input message tracked through G0–G18.
Provider observed in dispatch	`openai`	Provider recorded near the deeper reply-runtime boundary.
Model observed in dispatch	`gpt-5.5`	Model value recorded in the Gateway / dispatch trace.
Available tools	`NOT RECORDED`	R1 invokes zero tools; the complete available-tool inventory is not archived in the saved trace.
Python version	Not archived in the current trace	Record with `python --version` if Python scripts are part of the public artifact.
Node.js version	Not archived in the current trace	Record with `node --version`; needed for UI / Gateway replay scripts.
Package manager and lock file	Not archived in the current trace	Record `npm`, `pnpm`, or `yarn` version and include the matching lock file.
Operating system	Not archived in the current trace	Needed for reproducibility of local traces and timings.
CPU / GPU / RAM	Not archived in the current trace	Needed only if timing claims are made beyond this case study.
Browser / client build	Not archived in the current trace	Needed because the entry path starts from Control UI / WebChat.
LaTeX compiler	XeLaTeX	Document compiler.

B. Main Runtime Parameters

Parameter	R1 value	Role
SESSION_KEY	agent:main:dashboard:trace-82fbff4483c4	Links the request to the Dashboard Session and later to Agent main.
SESSION_ID	NOTRECORDED	Gateway Session identity is not emitted; AR-layer Session ID is separate.
RUN_ID	ANON-R1-RUN-ID	Used for idempotency, admission, and run-state tracking.
AgentId	main	Effective Agent resolved in G9 and confirmed again in G17.
ChatType	direct	Chat type carried in the runtime message context.
attachments	[]	No attachment path is exercised in this case.
resolverSource	default_getReplyFromConfig	Shows that G18 selected the default reply resolver.
provider	openai	Provider value observed near the deeper reply-runtime boundary.
model	gpt-5.5	Model value observed in the dispatch trace.
availableTools	NOT RECORDED	Available-tool inventory is not archived as an R1 trace value. R1 runtime evidence records tools invoked as 0.

C. Reproduction Parameters to Publish

Parameter group	Recorded value	What to publish
Repository URL	`ANONYMIZED-ARTIFACT-URL`	Anonymous artifact handle used during double-blind review; restore the public URL only after review if allowed.
Demo video	`ANONYMIZED-DEMO-URL`	Anonymous demo handle used during double-blind review; restore the public demo link only after review if allowed.
Source checkout	0790d9f593ad30c940ed93b5872a8cf6d6f3cf8c	Publish the exact commit, branch or tag, and any local patch status.
Trace input	`How to make a cake?`	Publish the exact request payload after removing secrets.
Authentication mode	Shared-token path accepted; role `operator`; scope `operator.write`	Publish the non-secret configuration needed to reproduce G0–G2.
Gateway route	`chat.send`	Publish route name, client entry point, and request schema version if available.
Runtime IDs	Session key and run ID recorded above; Gateway Session ID not recorded	Publish anonymized IDs and explain how they map to trace events.
Timing values	Case-specific observations only	Publish raw timestamps if making any performance claim.
Randomness / seeds	Not archived	Publish seed values if replay scripts include randomized steps.
Secrets and keys	Not included	Do not publish secrets. Replace them with redacted placeholders and document their role.

D. Trace and Measurement Scope

- The controlled cohort covers R1–R7, with R1 used as the detailed Gateway case. It does not benchmark answer quality.
- Runtime timing values should be treated as case-specific observations, not as performance claims.
- Branches with no direct runtime event are marked as source-derived in the stage traces.
- The full source trace is included in this appendix so that the main text can stay readable.

E. Anonymous Artifact Checklist

Artifact item	Status
Anonymous artifact link	`ANONYMIZED-ARTIFACT-URL`. Author-identifying repository URL is withheld during double-blind review.
Anonymous demo link	`ANONYMIZED-DEMO-URL`. Author-identifying video URL is withheld during double-blind review.
Trace logs	Saved trace artifacts contain the R1 G0–G18 runtime events and the post-G18 `agentRuntime` events where emitted.
Source snapshot	Pinned source claims use 0790d9f593ad30c940ed93b5872a8cf6d6f3cf8c.
Reproduction script	Replay and trace-inspection instructions are an artifact-packaging requirement, not an additional evidentiary claim in this appendix.
Environment manifest	Environment metadata is a release-packaging requirement. It is not used as runtime evidence unless a concrete field is recorded in the saved trace.
README	Reviewer inspection path: load the saved trace, inspect G0–G18, and reproduce the main figure.
Optional interactive artifact	Optional; not used as evidence in the reported counts.